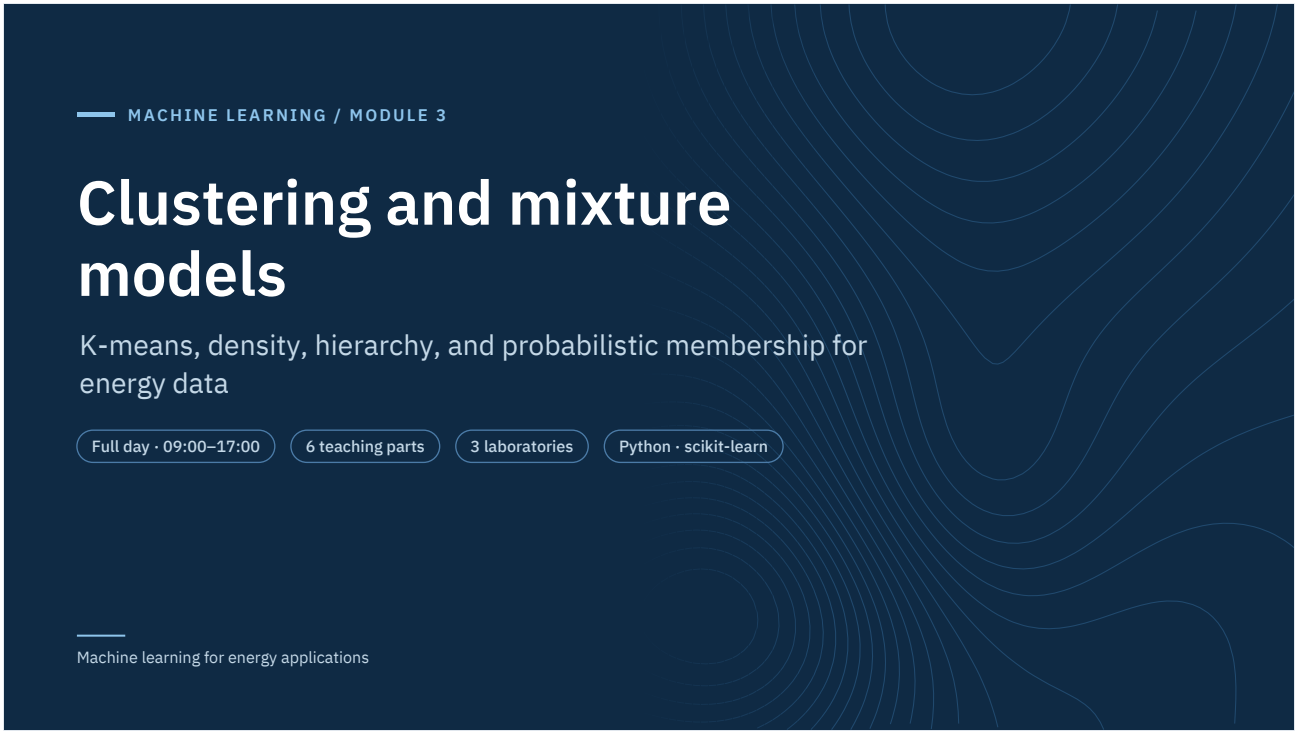
Clustering and mixture models

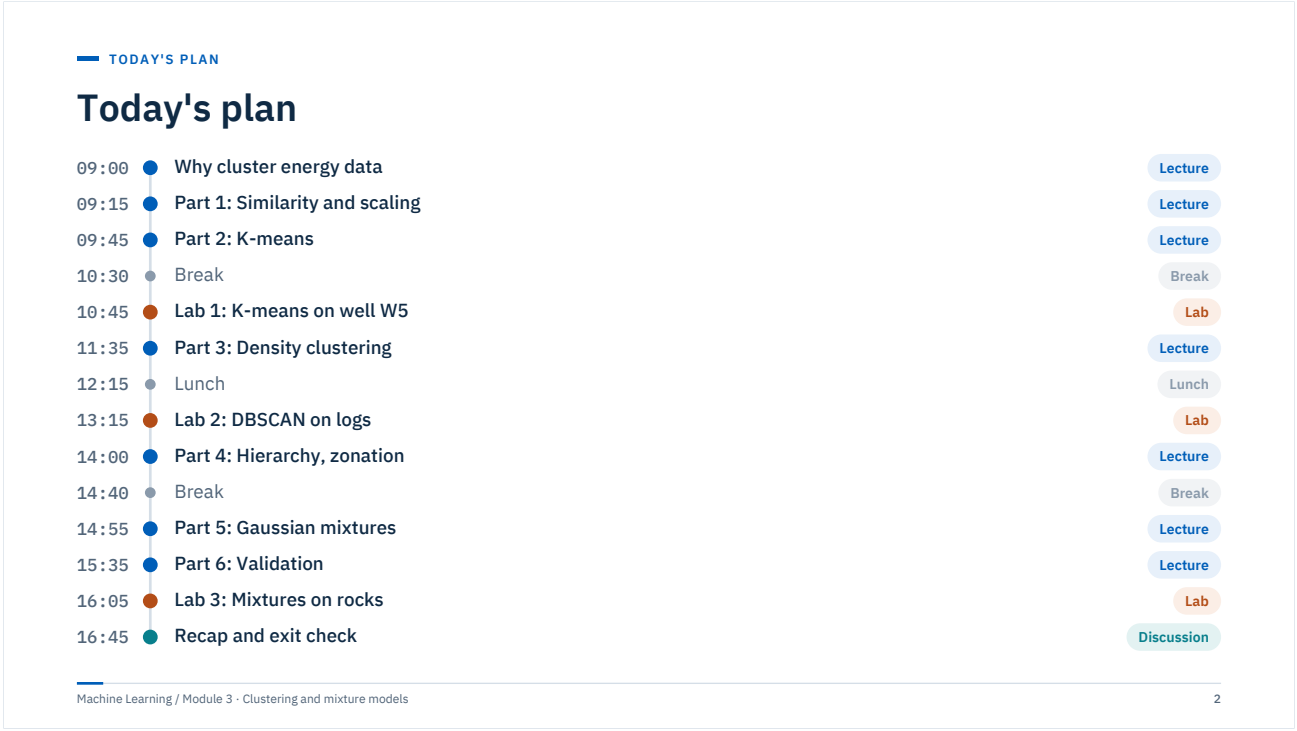


Welcome learners and set the frame for the day: clustering finds groups that nobody has labelled, so every result is a hypothesis about structure, not a discovery of truth. Prerequisites are distance, scaling, basic probability and array operations; the supervised-learning ideas from Module 1 (scaling inside pipelines, leakage) reappear here in unsupervised form.

The running example is a synthetic well, S-1, with four simulated lithologies. Because it is synthetic we know the answer, which lets us check methods honestly. The three labs use the supplied course files: well W5 logs, an SP–GR log sample, and a rock geochemistry table.

State the ground rule early and repeat it all day: a cluster ID is not a facies, a rock type or an operating state until external evidence supports that name. [2 min]

Today's plan

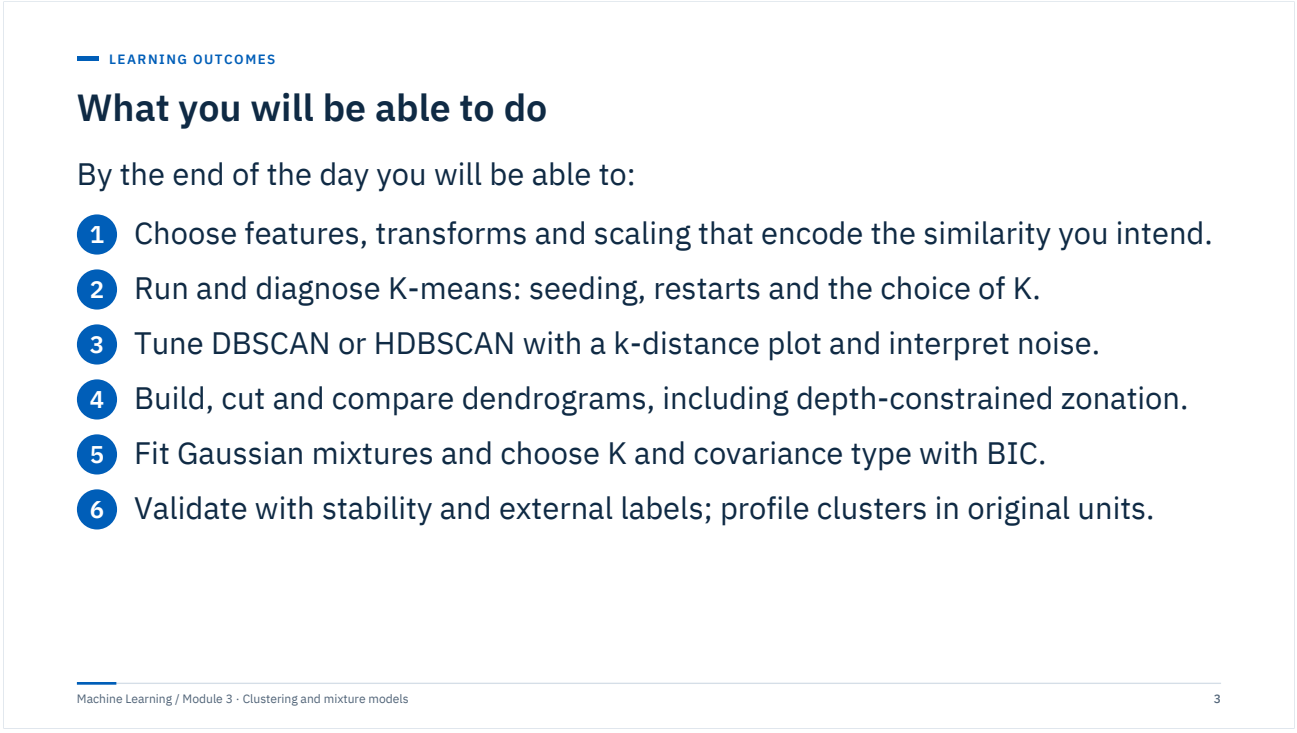


Walk through the day. Six teaching parts alternate with three labs so that each method is practised soon after it is taught: K-means before the first lab, density clustering before the second, and mixtures plus validation before the third.

Labs are timed including a ten-minute debrief each. If the morning runs late, shorten Part 3 by skipping the OPTICS slide; if the afternoon runs late, the method-comparison notebook (exercises/05_comparison) becomes take-home work.

Tell learners that every lab asks for the same kind of evidence: a plot, a table in original units, and one sentence stating a hypothesis and its limitation. That habit is the real deliverable of the day. [2 min]

What you will be able to do

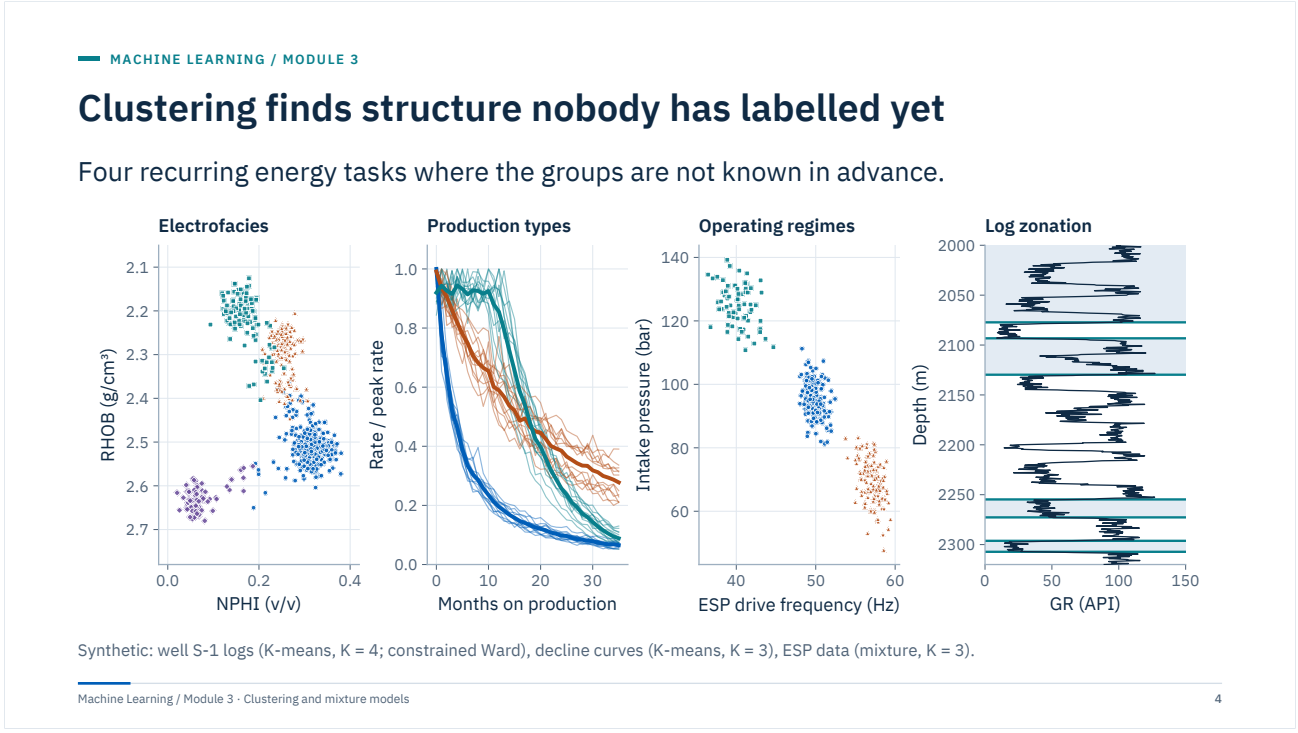


Read the outcomes aloud and point out that each is observable: learners will produce a plot, a number or a short written judgement for each one during the exercises and labs.

Outcome 1 is deliberately first. Most clustering failures in practice come from features and scaling, not from the choice of algorithm. Outcomes 2 to 5 cover the four families of methods used in the energy industry: centroid, density, hierarchical and model-based. Outcome 6 is what separates a useful grouping from a colourful plot.

Ask the room: who has used clustering on logs, production or sensor data before, and how did you decide the result was right? Collect two or three answers and return to them in Part 6. [2 min]

Clustering finds structure nobody has labelled yet

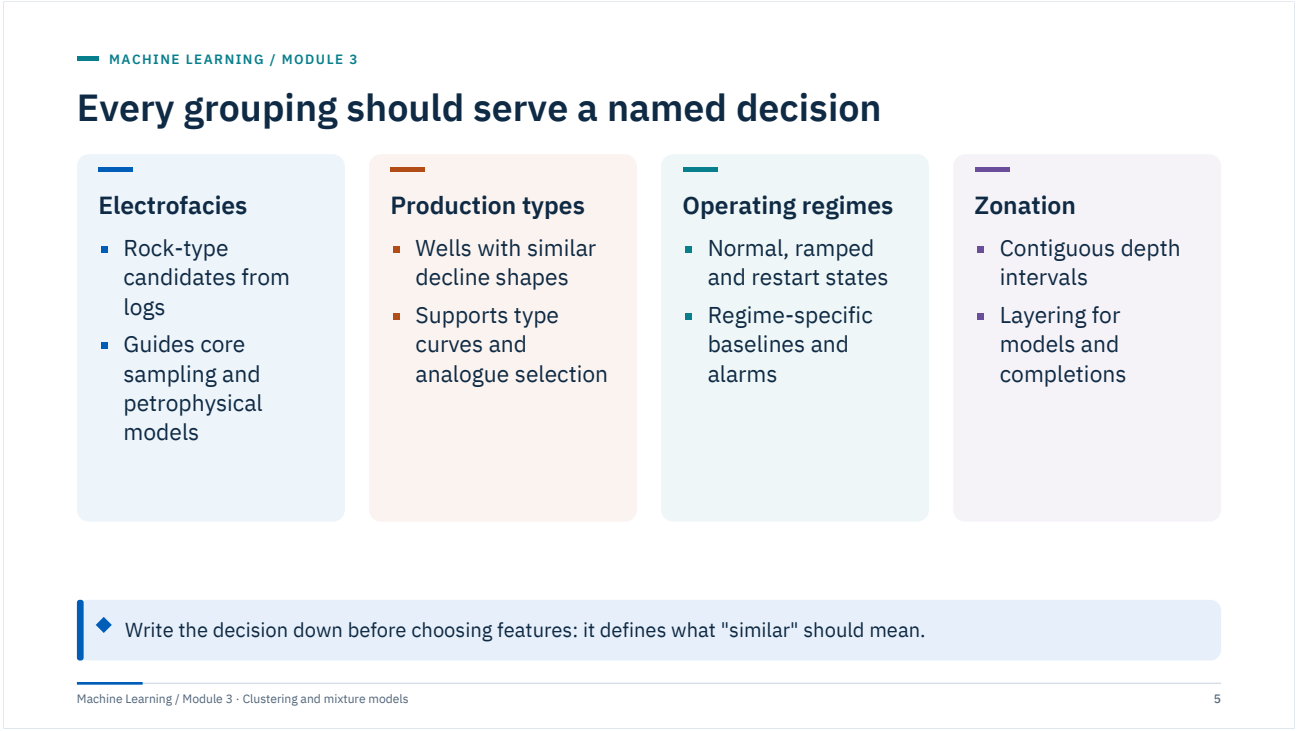


Tour the four panels. Electrofacies: clustering log responses (here neutron against density) to propose rock-type candidates where core is sparse. Production types: grouping normalized decline curves so that engineers can build type curves or pick analogues. Operating regimes: separating the states a pump or plant runs in (normal, ramped, restart) so that baselines and alarms can be regime-specific. Log zonation: splitting a well into contiguous depth intervals.

All four panels are computed on synthetic data with the methods named in the caption; they show what the output looks like, not field performance.

Point out the difference between the first three panels and the fourth: zonation respects depth order, the others ignore it. We will see in Part 4 that this is a constraint we add explicitly. [3 min]

Every grouping should serve a named decision



The same well data can be clustered in several valid ways depending on the decision. A completion engineer wants contiguous intervals; a petrophysicist wants recurring rock types; a reservoir modeller may want both at different scales.

Ask: for your own work, which decision would a grouping support, and who would act on it? Take one or two examples.

Emphasize that clustering is exploratory. It generates candidate groups that must then be checked against physical evidence such as core, cuttings, pressure tests or operating logs. A grouping that nobody can act on is not wrong, but it is not yet useful.

Transition: the decision determines the similarity measure, which is the subject of the warm-up question. [2 min]

Similarity is part of the problem definition

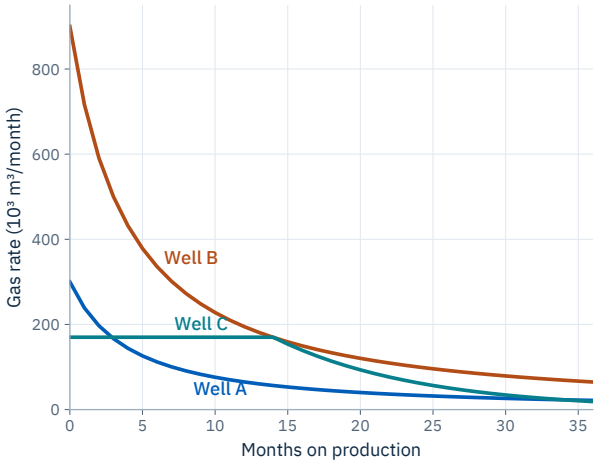
PRACTICE

5 min · Pairs

Similarity is part of the problem definition

You want to group wells with similar production behaviour.

1. Which well is most similar to well A, and why?
2. Should total production level affect similarity?
3. Which variables might simply encode well identity or age?



Synthetic decline curves: B is exactly 3 × A; C has a 14-month plateau.

Machine Learning / Module 3 · Clustering and mixture models 6

Give pairs three minutes, then collect answers. [5 min]

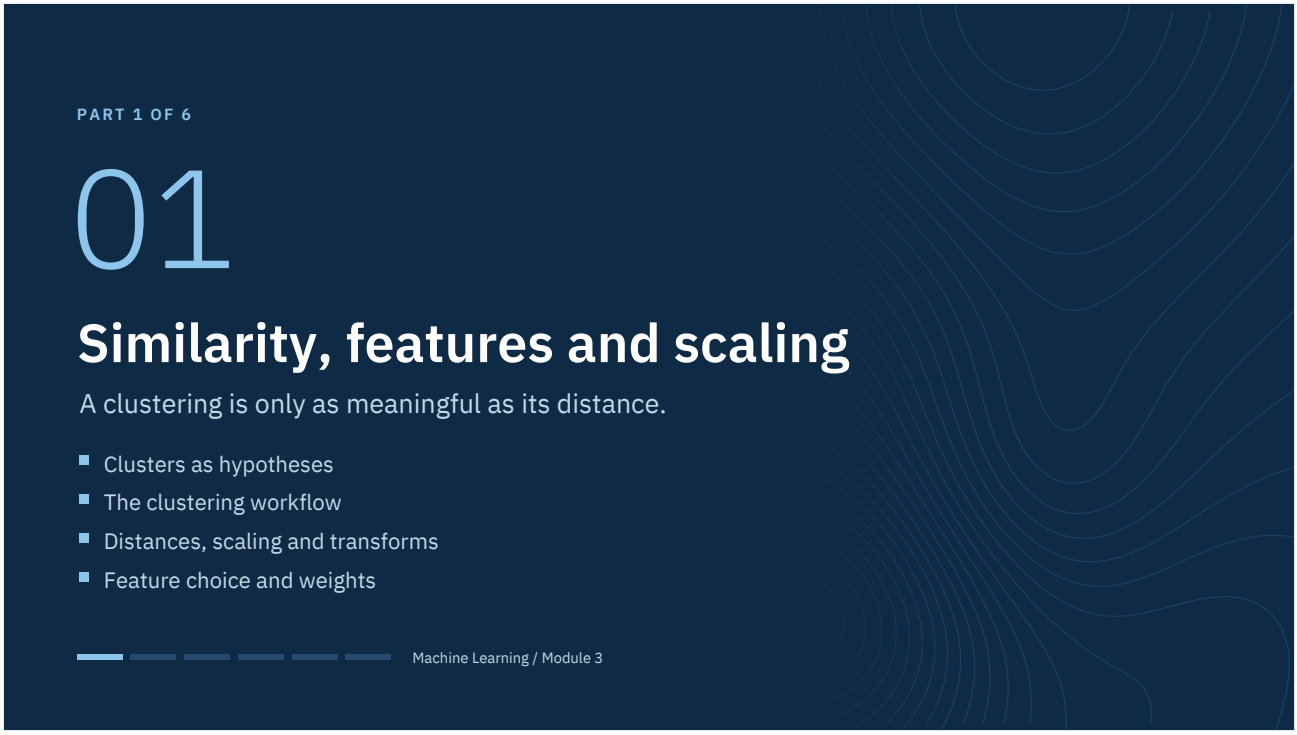
Worked answer. It depends on the representation, which is the point. On raw monthly rates the Euclidean distance from A to C is 381 and from A to B is 1,160 (units of 10^3 m^3 per month), so A is closest to C: magnitude dominates. If each profile is divided by its peak rate, A and B become identical (distance 0) while A–C is 2.90, so A is closest to B: shape dominates. The correlation of A and B is 1.00 and of A and C 0.67, which also groups A with B.

Should level matter? For reserves or facility sizing, yes; for decline mechanism or completion effect, usually no, so normalize.

Variables that encode identity or age: well name, spud date, first-production month, cumulative time on production. If they enter the features the clusters may simply rediscover field age or operator campaigns.

Common error: assuming there is a single correct metric. There is not; the metric follows the decision.

Similarity, features and scaling



This part is short but it carries more weight than any algorithm we will see. By the end learners should be able to predict how a change of units, a log transform or an extra feature will change a clustering result before running it.

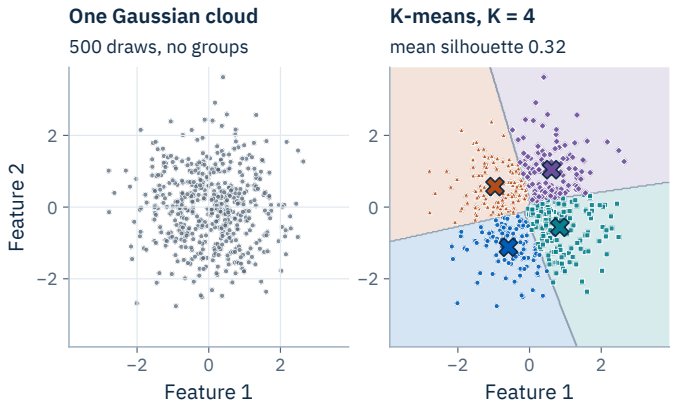
Link to Module 1: there, scaling was fitted inside cross-validation folds to avoid leakage. In clustering there is usually no held-out target, but the same discipline applies when a fitted scaler and model are reused on new wells: fit once, save, and transform new data with the saved objects. [1 min]

A cluster label is a hypothesis

SIMILARITY, FEATURES AND SCALING

A cluster label is a hypothesis

- Algorithms group by their objective and distance, not by physical truth
- Scaling, features and resolution change the groups
- Cluster numbers have no order or physical meaning
- Naming a cluster as a facies needs external evidence



Synthetic: 500 draws from one bivariate normal (seed 11); K-means, K = 4.

Machine Learning / Module 3 · Clustering and mixture models 8

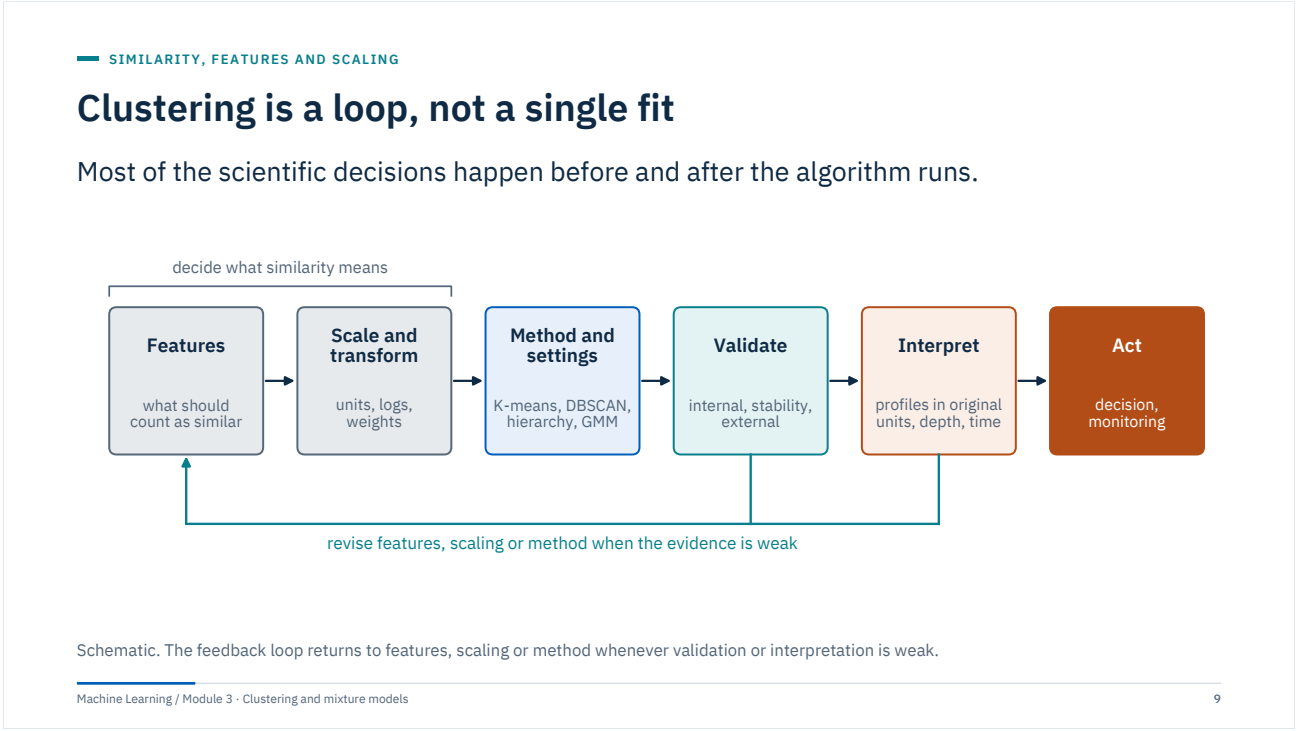
The left panel is a single Gaussian cloud: there are no groups. K-means with K = 4 still returns four tidy clusters with centroids and boundaries. Its mean silhouette is 0.32, and for K = 2 to 6 the value stays between 0.32 and 0.35, so a moderate silhouette is not evidence of structure on its own.

This is the central caution of the day: an algorithm will partition continuous data even when no discrete types exist. Log responses often vary continuously (for example with shale volume), so a partition can be a convenient discretization rather than a set of natural types.

Colouring a projection by cluster ID is a useful diagnostic, but it is not independent validation of those clusters, because the colours come from the same data.

Question for the room: what evidence would convince you that four groups exist here? Expected answers: a density with separated modes, stable clusters across resamples, and agreement with independent labels. [3 min]

Clustering is a loop, not a single fit



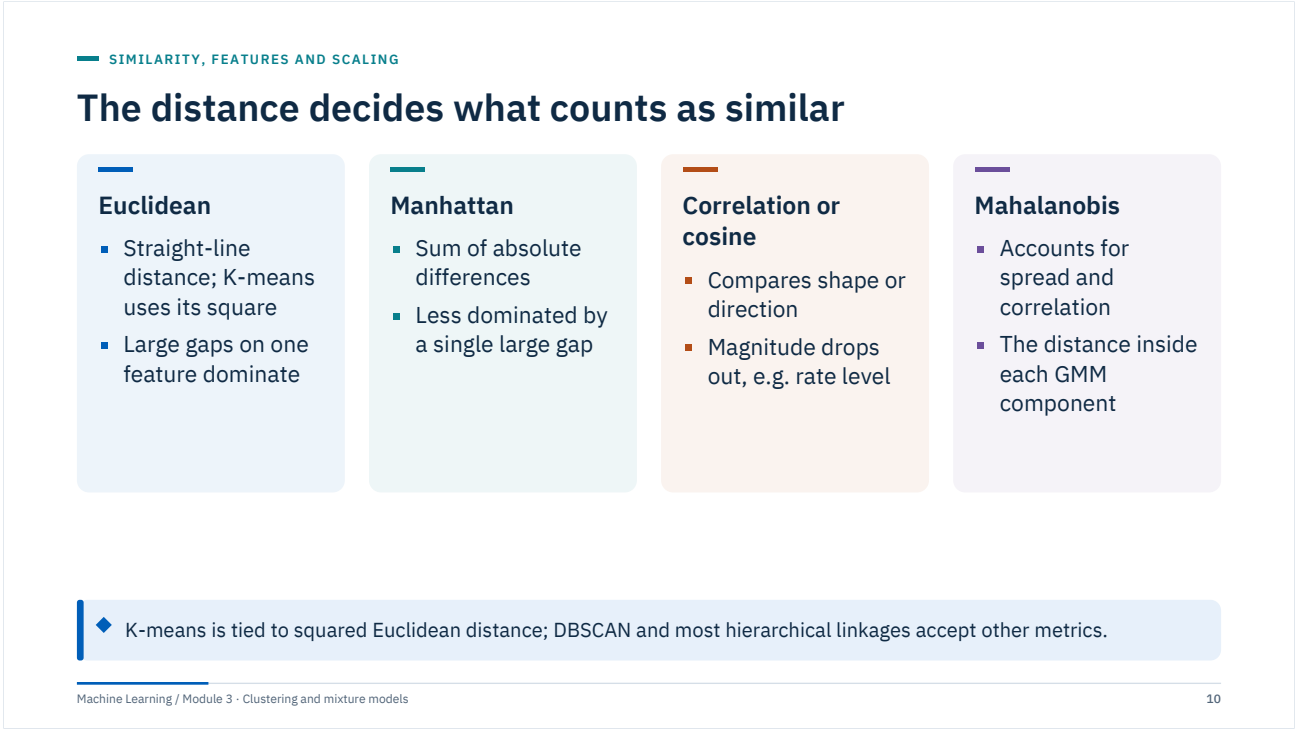
Walk left to right. Features: decide what should count as similar, following the decision from the opening. Scale and transform: units, logarithms, robust scaling, deliberate weights. Method and settings: the algorithm family and its parameters. Validate: internal indices, stability under resampling, and external labels where they exist. Interpret: profiles in original units, maps onto depth or time. Act: the decision, and monitoring if the grouping is used operationally.

The first two boxes decide what similarity means; the algorithm only optimizes within that choice. The loop back is normal practice, not failure: expect several passes.

Tie to the labs: every lab deliverable touches at least three boxes, typically scaling, validation and interpretation.

Transition: the next slides open the first two boxes, starting with the distance itself. [2 min]

The distance decides what counts as similar



Euclidean distance is the default and the only choice for K-means, because the centroid (a mean) is the point that minimizes squared Euclidean distance. Squaring means one large difference can dominate the total.

Manhattan (city-block) distance adds absolute differences and is less sensitive to a single extreme feature.

Correlation or cosine distance compares the shape or direction of a profile and ignores its level, which is what the peak-normalized decline curves did in the warm-up.

Mahalanobis distance rescales by the covariance, so correlated or high-variance directions count less. A Gaussian mixture component measures distance this way, which we will see in Part 5.

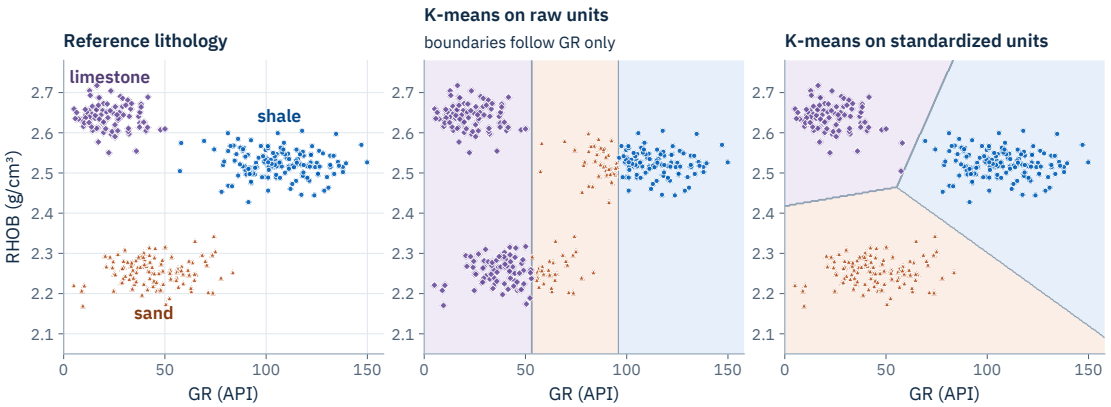
In scikit-learn, DBSCAN, HDBSCAN and AgglomerativeClustering take a metric argument, but Ward linkage requires Euclidean distance. Mixed numeric and categorical data need specialized distances (for example Gower), which are not built into scikit-learn. [3 min]

Feature scaling changes the clusters

SIMILARITY, FEATURES AND SCALING

Feature scaling changes the clusters

Same data and K: raw GR (SD 39 API) swamps RHOB (SD 0.16 g/cm³) until we rescale.



Synthetic GR and RHOB samples of three lithologies (seed 4); K-means, K = 3, n_init = 10; cluster colours matched to the reference.

Machine Learning / Module 3 · Clustering and mixture models 11

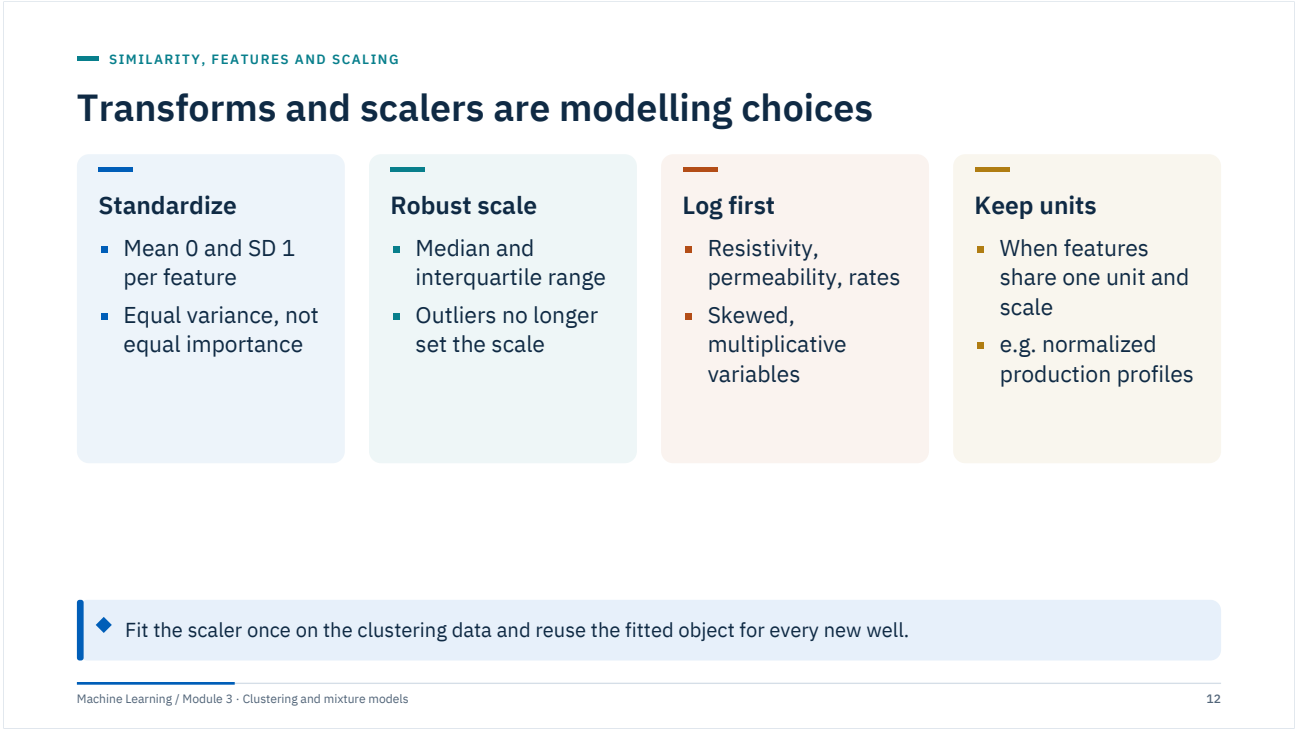
In raw units the standard deviation of gamma ray (39.5 API) is numerically about 250 times that of density (0.156 g/cm³), so squared gamma-ray differences dominate every distance. K-means therefore cuts the data into gamma-ray bands: the boundaries in the middle panel are vertical, and low-GR sands and limestones are mixed.

After standardizing each feature to unit variance, density can separate sand from limestone and the result matches the reference lithology. In this sample 65% of points fall in the matching group on raw units against 99% after standardizing (computed after best label matching).

Stress the general rule: in any distance-based method the feature with the largest numeric spread dominates unless you decide otherwise. Changing units (m to ft, bar to psi) changes the answer.

Question: what would happen if RHOB were recorded in kg/m³ (values near 2,500)? Answer: density would now dominate raw distances instead of gamma ray. [3 min]

Transforms and scalers are modelling choices



Standardization gives each feature the same variance, which gives each the same potential influence on distance. That is a choice, not neutrality: a low-variance log that mostly carries noise will be amplified to the same weight as an informative one.

Robust scaling uses the median and interquartile range, so washouts, spikes or bad-hole intervals do not compress everything else.

Resistivity spans orders of magnitude; clustering raw ohm·m values lets a few high readings dominate, so take log10 first. The same applies to permeability and often to rates.

Sometimes no scaling is right: normalized production profiles are already in comparable units, and standardizing each month separately would distort the shape.

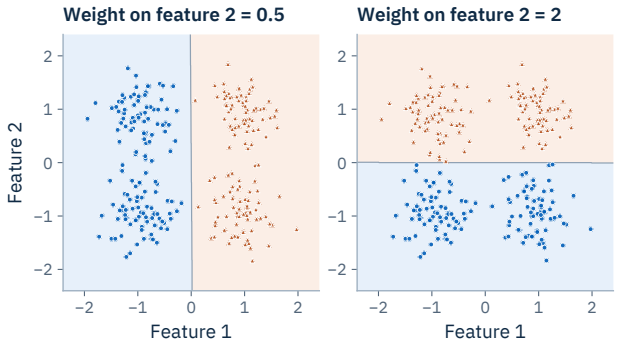
Pipelines matter: keep the transform and the model together (make_pipeline) and save both, so new data are transformed with the same fitted parameters. Refitting the scaler on each new batch silently moves the cluster boundaries. [3 min]

Feature choice and weights set the answer

SIMILARITY, FEATURES AND SCALING

Feature choice and weights set the answer

- Each feature's influence grows with its spread after scaling
- A duplicated or strongly correlated log counts twice
- Depth, well ID and dates encode location, not rock or behaviour
- Weight deliberately: multiply a standardized column by w



Synthetic four-group data (seed 8), standardized; K-means, K = 2, with feature 2 multiplied by the stated weight.

Machine Learning / Module 3 · Clustering and mixture models 13

The data contain four groups at the corners of a rectangle. With K = 2 there are two almost equally good splits. Halving the weight of feature 2 makes K-means split left and right; doubling it makes K-means split top and bottom. Nothing about the data changed, only our statement of what matters.

In practice weights appear implicitly. Adding both a density porosity and a neutron-density crossplot porosity is like doubling the weight of porosity. Including depth or pressure (which tracks depth) makes clusters partly spatial: they group intervals by where they are rather than by what they are. We will see this with real data in Lab 1.

Deliberate weighting is legitimate if you can defend it: for example, down-weighting a noisy log, or up-weighting the log that best separates the rock types of interest in core.

Transition to the exercise: choosing features for an electrofacies study. [3 min]

Choose the features for an electrofacies study

PRACTICE

5 min Pairs

Choose the features for an electrofacies study

Available for 12 wells: GR, RHOB, NPHI, RT, DT, CALI, DEPTH and well name.

1. Which logs would you cluster on, and which would you leave out?
2. Which need a transform before scaling?
3. What would you check before trusting the clusters?

Machine Learning / Module 3 · Clustering and mixture models 14

Give pairs four minutes, then take answers. [5 min]

Worked answer. Use GR, RHOB, NPHI, log10(RT) and DT, which respond to lithology, porosity and fluid. Leave out DEPTH and well name: they encode location and would make clusters partly spatial or well-specific. Leave CALI out of the features but use it for quality control: washed-out hole corrupts density and neutron readings, so flag or remove bad-hole intervals before clustering.

Transforms: log10 of resistivity; consider robust scaling if spikes remain after quality control. RHOB and NPHI are strongly related, so together they give porosity extra weight; that may be acceptable, but say so.

Checks before trusting clusters: distributions per well (is one well offset by a tool or calibration difference?), whether clusters simply follow wells (a crosstab of cluster by well), stability under resampling by well, and a comparison with any cored interval.

Common error: including DEPTH "because it is available". Another: forgetting environmental corrections so that a borehole effect becomes a cluster.

K-means



K-means is the workhorse: fast, simple, and the first method most teams try. It is also the easiest to misuse because it always returns exactly K convex groups.

Define the vocabulary now. K is the number of clusters, chosen by the user. A centroid is the mean of the observations assigned to a cluster, in the transformed and scaled feature space; it is not necessarily a physically realizable sample. Assignment means attaching each observation to its nearest centroid. Inertia is scikit-learn's name for the objective value, the within-cluster sum of squared distances. [1 min]

K-means minimizes within-cluster squared distance

K-MEANS

K-means minimizes within-cluster squared distance

$$J = \sum_{i=1}^{n} \| x_i - \mu_{c_i} \|^2$$

1. Choose K initial centroids
2. Assign each observation to its nearest centroid
3. Move each centroid to the mean of its assigned observations
4. Repeat until the assignments stop changing

◆ Neither step can increase J, so the loop converges, but only to a local minimum.

Machine Learning / Module 3 · Clustering and mixture models 16

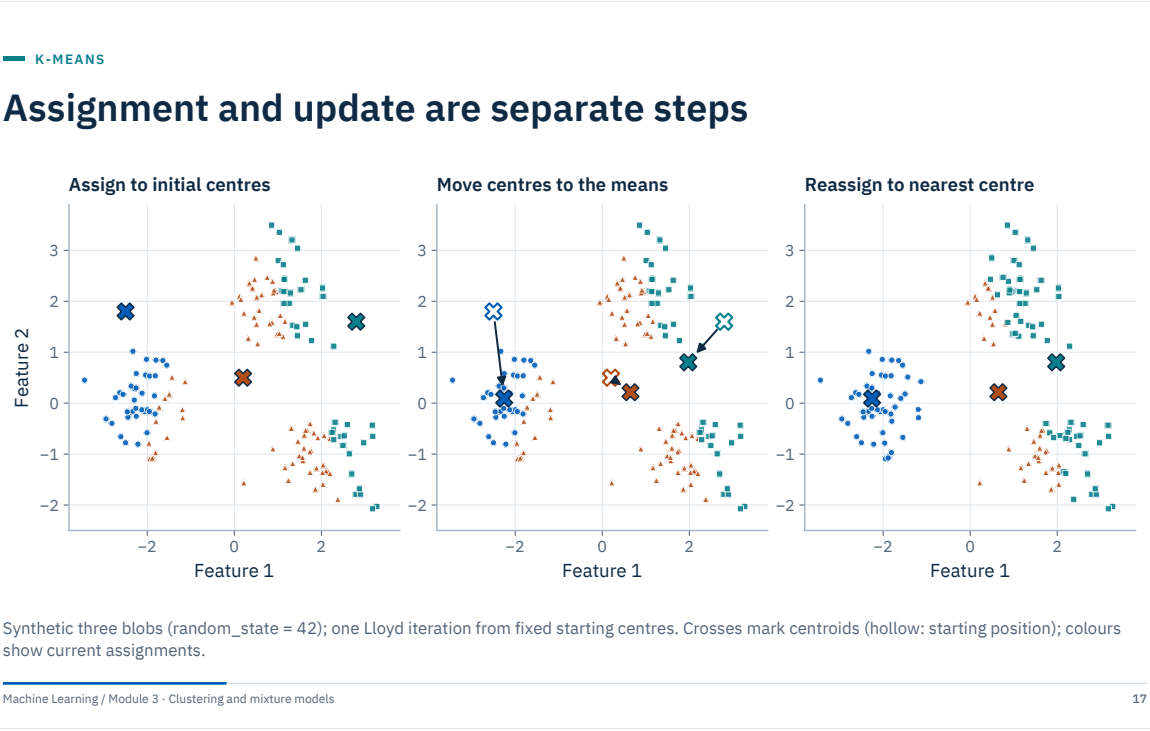
Define the symbols: x_i is observation i, c_i is the cluster it is assigned to, and μ_c_i (mu sub c_i) is that cluster's centroid. J (inertia) adds the squared distances from every observation to its own centroid.

Why the loop converges: with centroids fixed, assigning each point to its nearest centroid cannot increase J; with assignments fixed, the mean is the point that minimizes the sum of squared distances, so moving the centroid to the mean cannot increase J either. J therefore decreases or stays the same at every step, and there are finitely many assignments, so the loop stops.

It stops at a local minimum, which depends on the starting centroids. That is why initialization (k-means++) and several restarts (n_init) matter; set n_init explicitly for reproducible work.

Scikit-learn's KMeans uses this alternating scheme (Lloyd's algorithm) by default. [3 min]

Assignment and update are separate steps



Walk through the same observations across the three panels. The points never move; only the assignments and the centres change.

Panel 1: assignments to three arbitrary initial centres. Note that the orange centre in the middle collects points from all three visible groups. Panel 2: each centre moves to the mean of its assigned points; the arrows start at the old positions (hollow crosses). Panel 3: points are reassigned to the new nearest centre.

Ask learners to point out which centre moves most and connect the direction of the move to the points it had collected. The blue centre moves most (about 1.7 units): it started above the left group and drops to that group's mean. The teal centre moves down and left (about 1.2) towards the points it collected from two groups. The orange centre moves least (about 0.5) because its points surround it on all sides.

Transition: now do one update by hand on a line, which makes the arithmetic explicit. [2 min]

One update by hand

PRACTICE

4 min Individual

One update by hand

Observations on a line: 1, 2, 3, 8, 9, 10. Initial centroids: 2 and 8.

1. Assign each observation to a centroid.
2. Calculate the updated centroids.
3. Calculate the within-cluster sum of squares before and after the update.

Give three minutes, then work through on the board. [4 min]

Worked answer. Assignments: 1, 2 and 3 go to centroid 2; 8, 9 and 10 go to centroid 8. Updated centroids are the means: (1+2+3)/3 = 2 and (8+9+10)/3 = 9.

Sum of squares with the initial centroids: left cluster $(1-2)^2 + 0 + (3-2)^2 = 2$; right cluster $0 + (9-8)^2 + (10-8)^2 = 1 + 4 = 5$; total 7. After the update: left 2, right $1 + 0 + 1 = 2$; total 4. The objective fell from 7 to 4, as the previous slide promised. The next assignment step changes nothing, so the algorithm has converged.

Extension question: add an extreme observation at 100. It joins the right cluster, whose mean becomes (8+9+10+100)/4 = 31.75. At the next assignment 8, 9 and 10 are closer to 2 than to 31.75, so they move left; the final solution is {1, 2, 3, 8, 9, 10} with centroid 5.5 and {100} alone. One outlier captured a whole centroid.

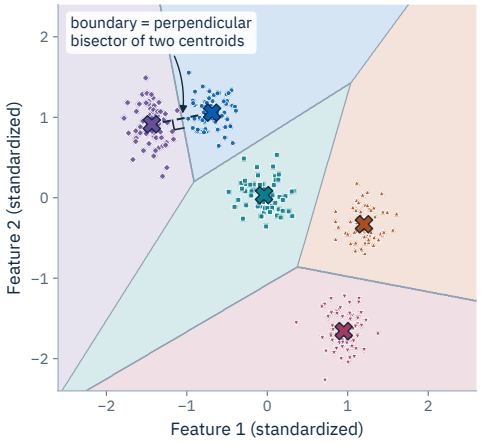
Common error: averaging the centroids instead of the observations.

Assignments carve the space into Voronoi cells

K-MEANS

Assignments carve the space into Voronoi cells

- Each point joins its nearest centroid
- Boundaries are perpendicular bisectors between centroids
- Every cell is convex: one cluster cannot curve around another
- New samples are assigned by the same rule with `predict`



Synthetic blobs (random_state = 6), standardized; K-means, K = 5. Shading marks each centroid's cell.

The set of locations closer to one centroid than to any other is that centroid's Voronoi cell. Between two neighbouring centroids the boundary is the line of points at equal distance from both, which is the perpendicular bisector of the segment joining them, marked with the dashed line and the right angle.

Consequences worth stating. Boundaries are straight lines (planes in more dimensions), and every cell is convex. So K-means cannot represent a crescent, a ring, or one group wrapped around another: it will cut such shapes with straight lines, as we will see with the two moons.

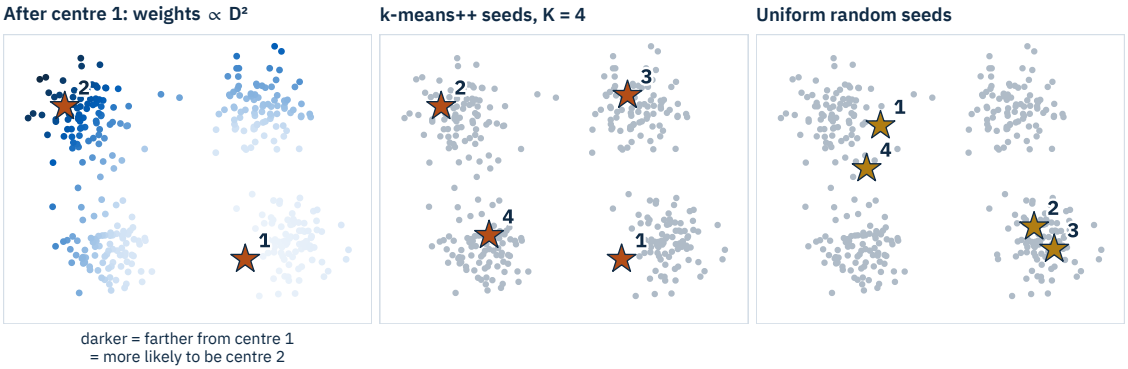
The same geometry defines prediction. A fitted KMeans assigns a new sample by finding its nearest centroid, so a new well can be labelled with predict after the same scaling. Samples far from every centroid still receive a label; K-means has no notion of "none of the above", unlike DBSCAN noise later. [2 min]

k-means++ spreads the initial centroids

K-MEANS

k-means++ spreads the initial centroids

Each new centre is sampled with weight D^2: squared distance to the nearest centre.



Synthetic four blobs (random_state = 2); one draw each. One seed per blob in 66% of 2,000 textbook k-means++ draws, 95% with scikit-learn's greedy variant, 9.5% for uniform sampling (exact).

Left panel: the first centre is chosen uniformly at random. Every other point then gets a weight equal to its squared distance to the nearest chosen centre; darker points are more likely to be chosen next. Middle panel: after four steps the seeds fall one per blob. Right panel: four uniformly chosen seeds put two in one blob and two in another, leaving two blobs without a seed.

The k-means++ numbers in the caption come from 2,000 repeated draws on these data: textbook k-means++ placed one seed in every blob 66% of the time. Four distinct points drawn uniformly manage this only 9.5% of the time; with four blobs of 90 points out of 360 the exact value is $4! \times 90^4 / (360 \times 359 \times 358 \times 357)$. Scikit-learn's kmeans_plusplus tries several candidates for each new centre and keeps the one that most reduces inertia, which raised the rate to 95%.

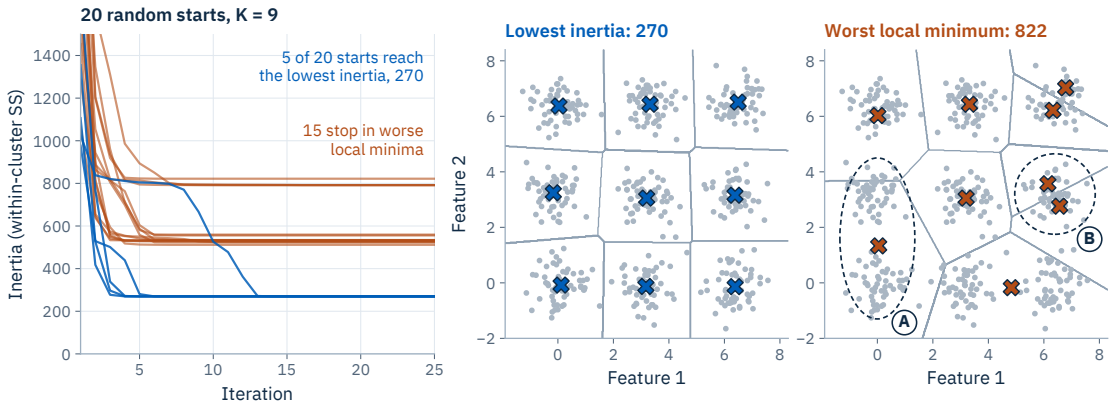
Seeding does not replace Lloyd iterations; it only gives them a better start. The method is from Arthur and Vassilvitskii (2007). [3 min]

Random starts end in different local minima

K-MEANS

Random starts end in different local minima

Lloyd iterations only go downhill; where they stop depends on where they start.



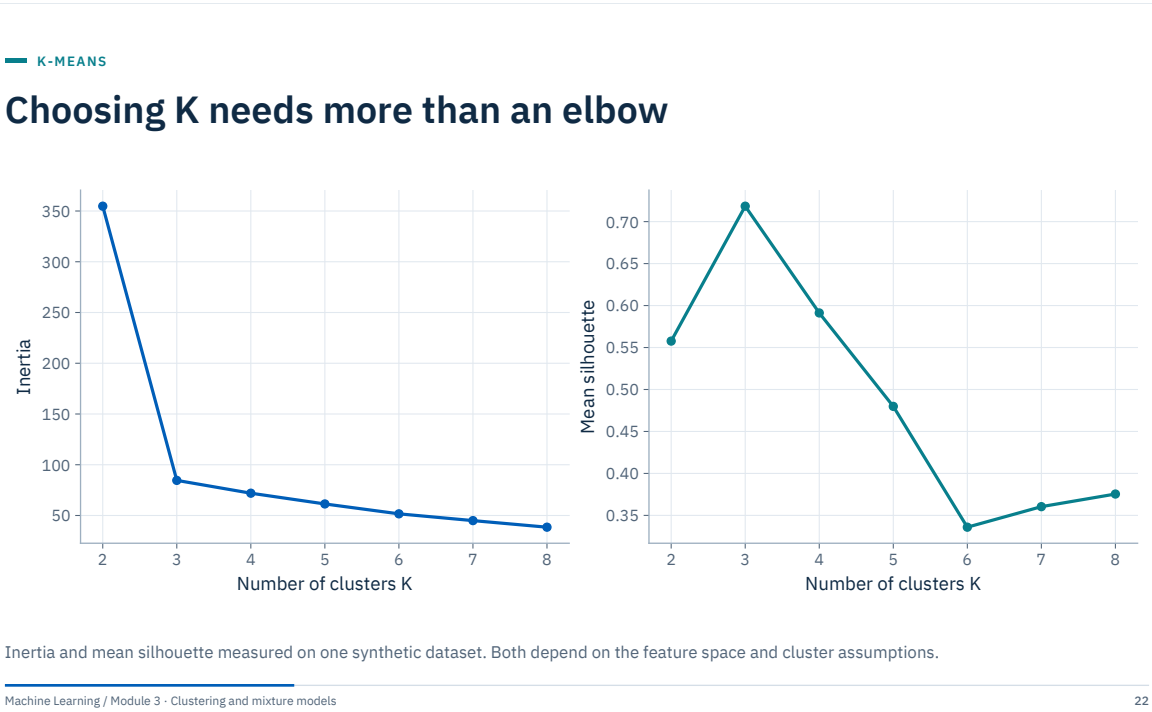
Synthetic 3 × 3 blob grid; 20 uniform random starts, K = 9. A: one centroid covers two blobs. B: two centroids split one blob.

Left: inertia against iteration for 20 random starts. Every curve decreases, as promised, but only 5 of 20 reach the lowest value (270); the other 15 stop on plateaus between about 510 and 820.

Middle: the lowest-inertia solution, one centroid per blob. Right: the worst run. At A a single centroid sits between two blobs and serves both; at B two centroids share one blob. Neither can escape by a single assignment-and-update step, which is exactly what a local minimum means. The worst run has one more defect of each kind (the bottom-middle and bottom-right blobs share a centroid; the top-right blob has two); ask learners to find them.

The remedy is restarts: run several initializations and keep the lowest inertia. In scikit-learn this is n_init. Since version 1.4 the default is n_init = "auto", which runs k-means++ only once, so set n_init explicitly. On these data a single k-means++ start reached the lowest inertia in 98.5% of 200 runs, against 5 of 20 uniform random starts here (and 22% of 200 runs with scikit-learn's init="random"). [3 min]

Choosing K needs more than an elbow



Inertia always falls as K rises, because more centroids can only reduce distances when solutions are optimal (local solutions can occasionally break this slightly). So lower inertia alone cannot choose K. The elbow is the K after which the gain flattens; here inertia drops from 355 at K = 2 to 85 at K = 3, then only to 72 at K = 4.

Mean silhouette peaks at K = 3 (0.72) and falls to 0.59 at K = 4. Here both agree, because the data are three well-separated compact blobs.

In real logs the elbow is often gradual and the silhouette curve flat, and the two may disagree. Treat them as two pieces of evidence alongside stability under resampling (Part 6) and domain plausibility.

Ask: if inertia and silhouette disagree, which would you trust? Expected answer: neither alone; check stability and what the extra cluster represents in original units. [2 min]

Silhouette compares cohesion with separation

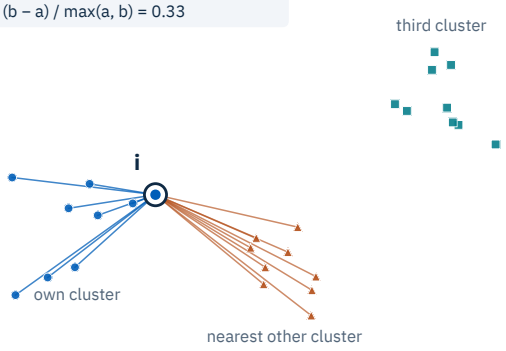
K-MEANS

Silhouette compares cohesion with separation

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

- a_i: mean distance to the other members of its own cluster
- b_i: smallest mean distance to the members of another cluster
- Near 1: well placed; near 0: on a boundary; negative: closer to another cluster

a = 1.53 mean distance to own cluster
b = 2.27 mean distance to nearest other
s = (b − a) / max(a, b) = 0.33



Synthetic three-cluster example (seed 21); values checked against scikit-learn silhouette_samples.

Machine Learning / Module 3 · Clustering and mixture models 23

Work through the picture. The ringed point i belongs to the blue cluster. Blue lines connect it to the other members of its own cluster; their mean length is a = 1.53. Orange lines connect it to the nearest other cluster; their mean length is b = 2.27. The third cluster is farther away, so it does not enter b. Then s = (2.27 − 1.53) / 2.27 = 0.33.

Interpretation: s is close to 1 when the point is much closer to its own cluster than to any other, near 0 when it sits between clusters, and negative when another cluster is closer on average, suggesting a competing assignment.

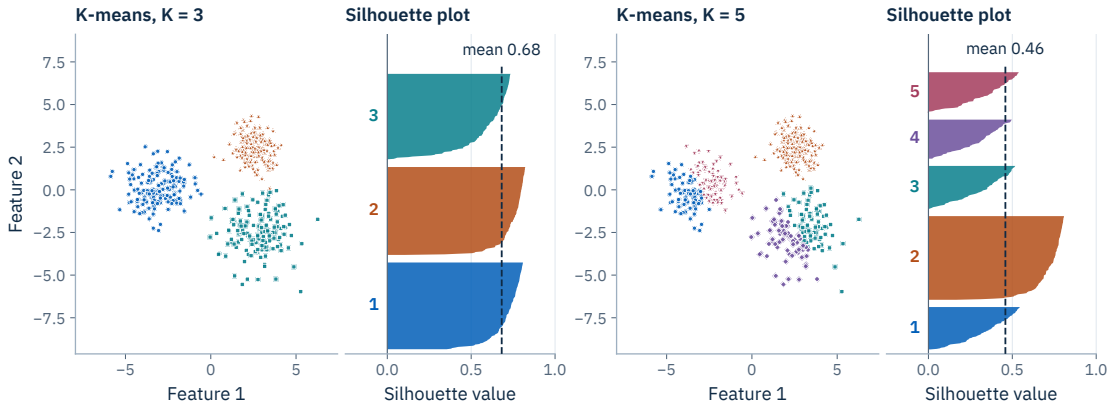
Technical notes. Silhouette is defined for 2 ≤ K ≤ n − 1 clusters; scikit-learn sets the value of a singleton to 0. For DBSCAN results decide explicitly how to treat noise (usually exclude it) and report the noise fraction with the score. Computing it needs all pairwise distances, so subsample large datasets. [3 min]

A silhouette plot shows every observation

K-MEANS

A silhouette plot shows every observation

Read each knife's width (size), length (fit) and tail (low or negative values).



Synthetic three-blob data (random_state = 5); K-means with K = 3 and K = 5; per-sample values from silhouette_samples.

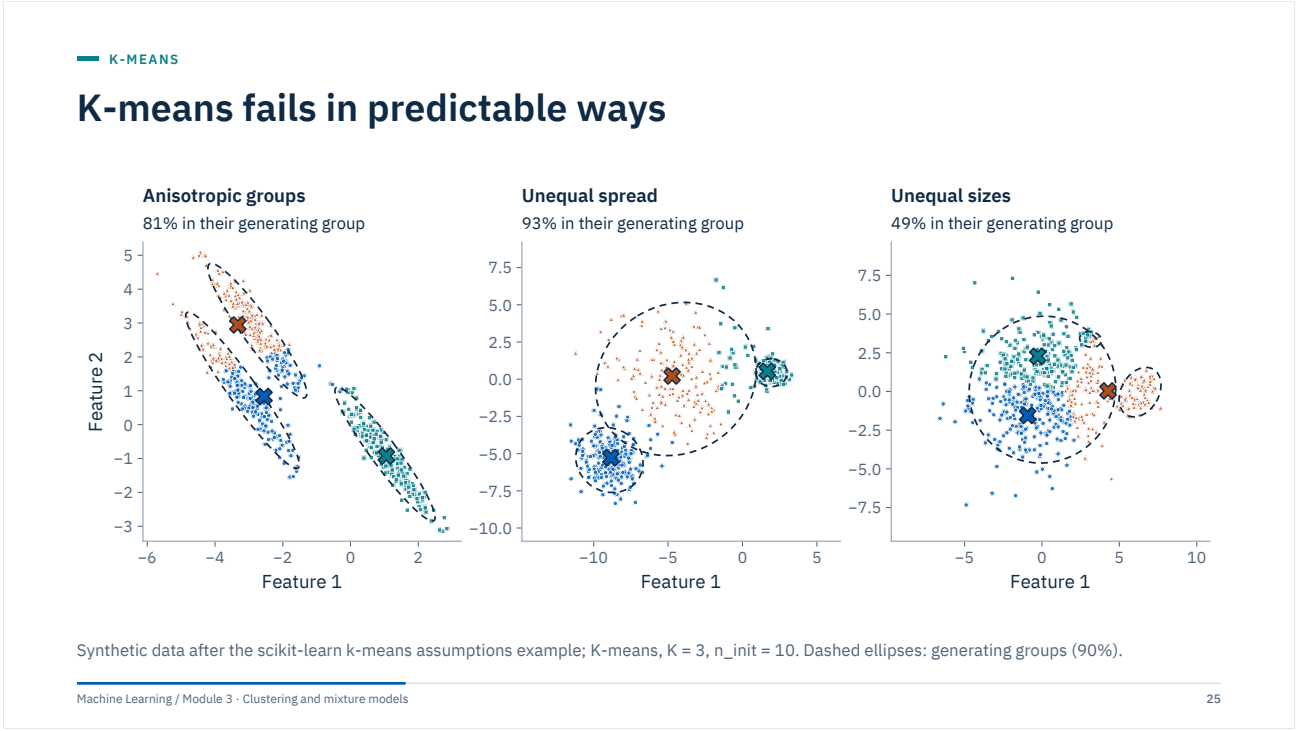
Each horizontal band is one cluster; within it, samples are sorted by silhouette value. The dashed line is the mean.

K = 3: three thick knives, all well above zero; the mean is 0.68. K = 5: the left blob and the lower-right blob have each been split in two. The new knives are thinner, most values fall below the K = 3 mean, and the mean drops to 0.46. Only the untouched cluster 2 keeps long values.

What to look for in practice: clusters whose knife never reaches the mean (a weak cluster), long negative tails (points probably misassigned), and very uneven widths (one giant cluster absorbing others).

The Lab 1 notebook asks for these plots for K = 2 to 6. Suggest learners put them side by side with the elbow plot and write one sentence per K. [2 min]

K-means fails in predictable ways



Three violations of the implicit K-means model, each with the correct K.

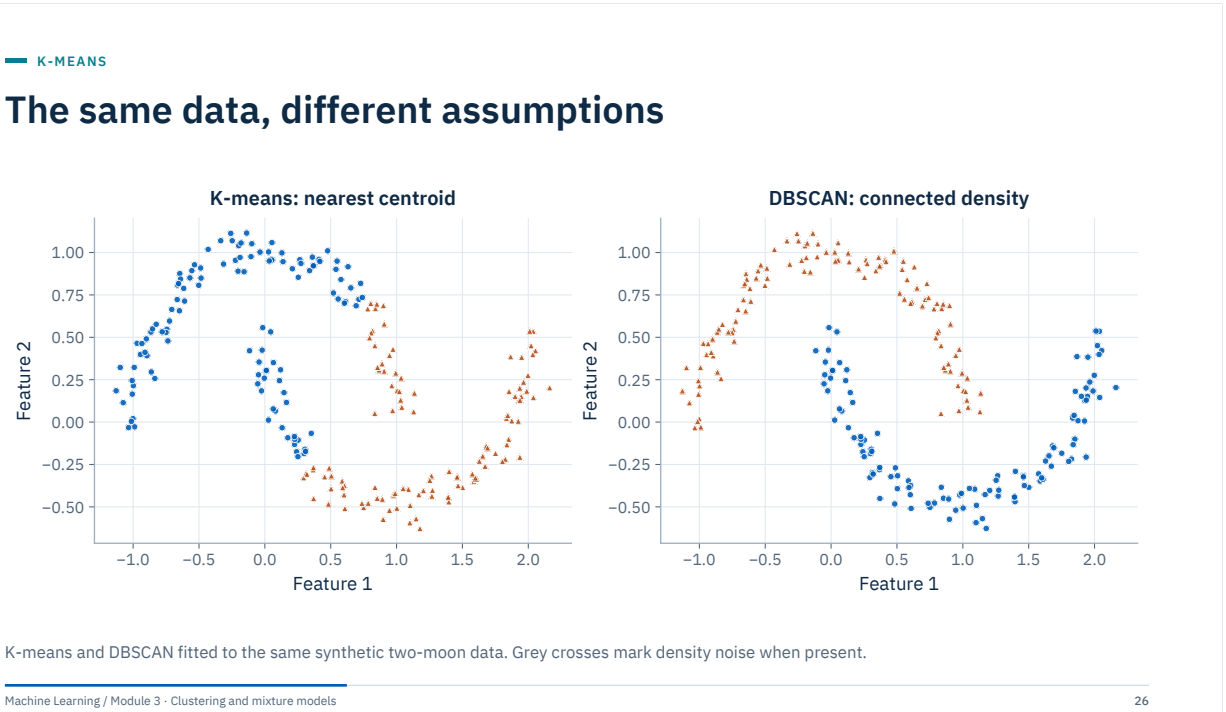
Anisotropic groups: elongated, parallel clusters. K-means prefers round groups, so it cuts across the elongated ones: 81% of points land in their generating group.

Unequal spread: one wide group between tighter ones. K-means boundaries sit roughly halfway between centroids, so the wide group loses its edge points to its tighter neighbours (93%).

Unequal sizes: a large diffuse group with a small and a medium group nearby. K-means spends two centroids splitting the large group, and the third covers the medium group plus part of the large one; the small group is absorbed. Only 49% are in their generating group.

Energy analogue: thin, high-contrast beds (for example coals or tight streaks) are small groups next to large shale populations; K-means tends to absorb them. On these data a full-covariance Gaussian mixture recovers the first two cases almost perfectly (100% and 99%) but not the third (59%); HDBSCAN with min_cluster_size = 10 isolates the small group and calls much of the large group's diffuse fringe noise. [3 min]

The same data, different assumptions

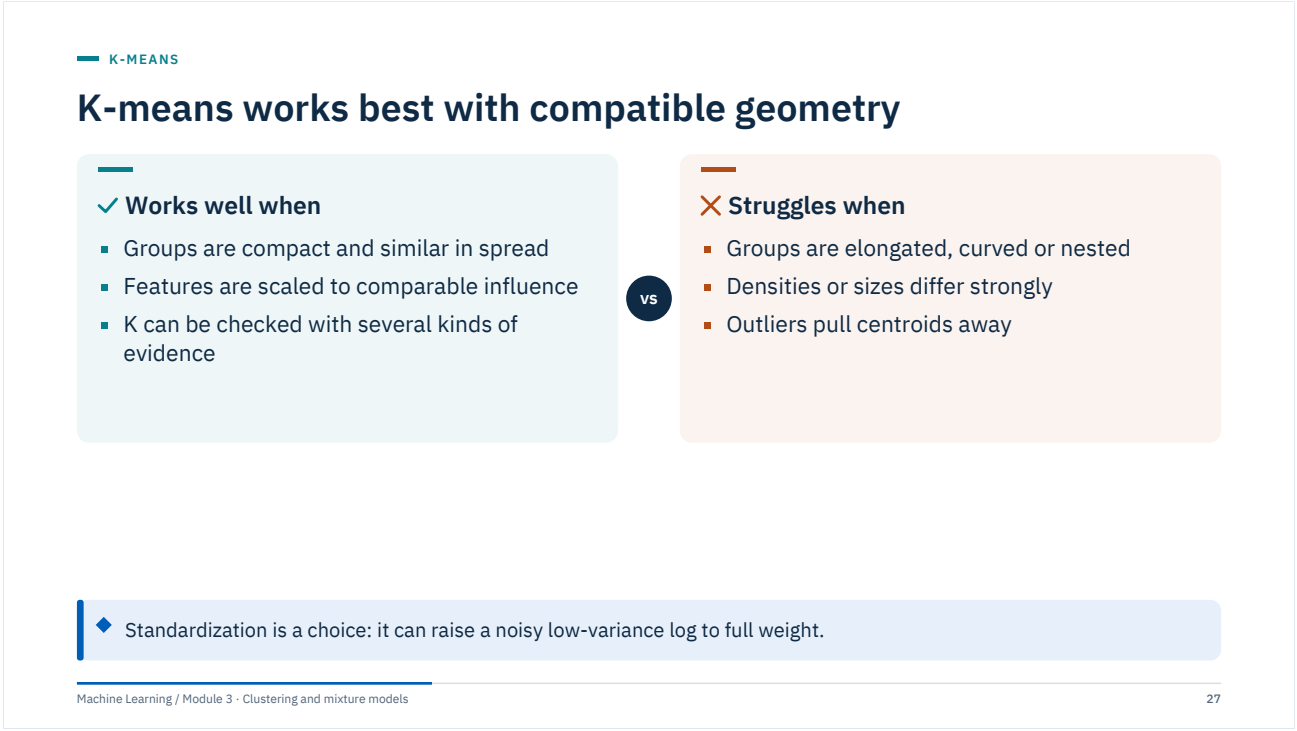


Ask why K-means cuts across each curved group. Its nearest-centroid assignment produces convex Voronoi cells, so the boundary between two clusters must be a straight line; no straight line separates two interleaved moons.

DBSCAN connects dense regions through chains of nearby points, so it can follow curved shapes, but only when its neighbourhood settings match the sampling density. With a poor setting it would merge the moons or break them apart, which we will study in Part 3.

This is the motivating case for the next part, and a reminder that a better score for one method on one dataset does not make it better in general: on compact blobs K-means is faster and at least as good. [2 min]

K-means works best with compatible geometry

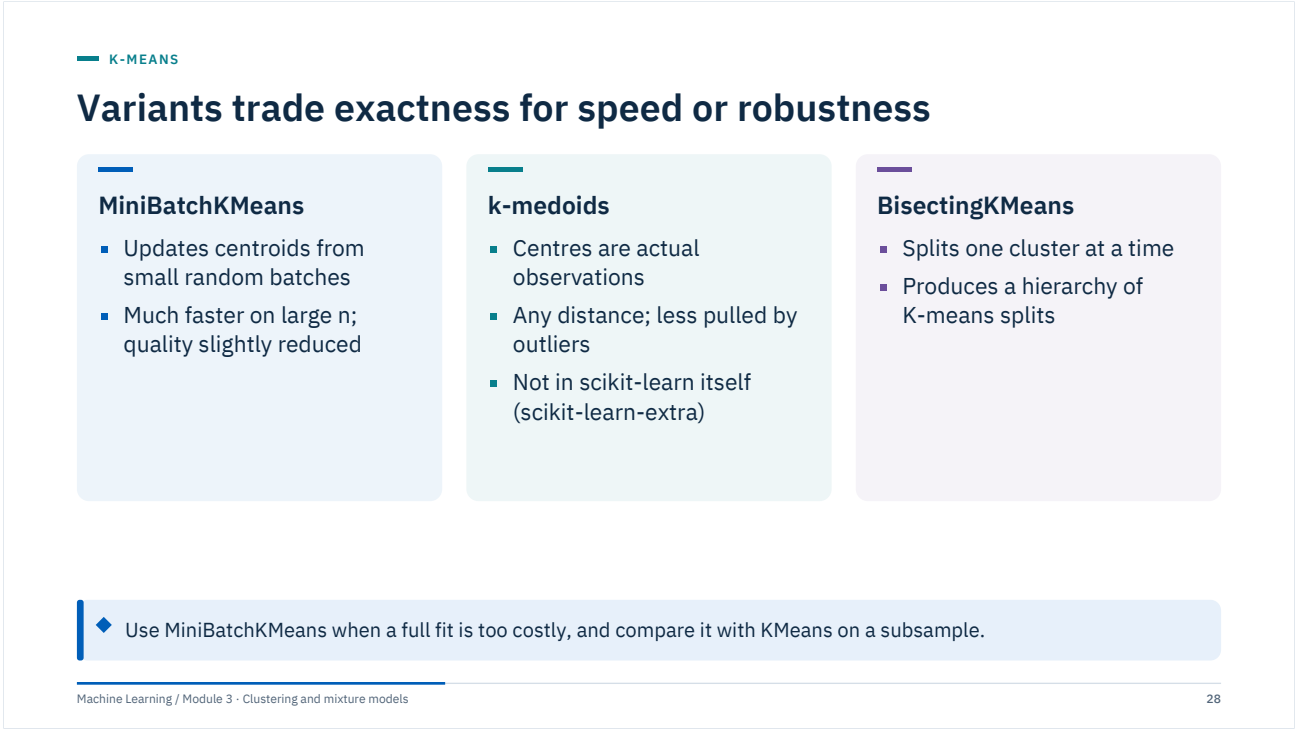


Summarize the geometry. K-means fits compact, roughly spherical groups of similar size in a space where Euclidean distance is meaningful. Standardize features when units should have comparable influence, and then examine sensitivity to outliers, unequal densities and initialization.

Scaling is a modelling choice, not a ritual. If a small-variance sensor carries important signal, standardization helps; if it mainly carries noise, standardization amplifies that noise to the same weight as an informative log.

For outliers, options include robust scaling, removing flagged bad-hole data before clustering, or using k-medoids (next slide). For non-convex shapes, prefer density or hierarchical methods; for elongated overlapping groups, a Gaussian mixture. [2 min]

Variants trade exactness for speed or robustness



MiniBatchKMeans (scikit-learn) updates centroids from random mini-batches, so it scales to millions of rows, such as high-frequency sensor data. The scikit-learn documentation notes that result quality is reduced compared with full KMeans, though in practice the difference can be small; check the inertia on a subsample.

k-medoids restricts each centre to be a real observation (a medoid). That makes it usable with any distance, including correlation or dynamic time warping, and less sensitive to outliers because a medoid cannot be dragged into empty space. The cost grows faster with n. It is available in the separate scikit-learn-extra package, not in scikit-learn itself.

BisectingKMeans (scikit-learn 1.1 and later) repeatedly splits one cluster in two (by default the one with the largest inertia), producing a hierarchy of K-means solutions; it is efficient when K is large because each split works on a subset of the data.

Mention these briefly; the labs use plain KMeans. [2 min]

A reproducible K-means fit

K-MEANS

A reproducible K-means fit

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

model = make_pipeline(
    StandardScaler(),
    KMeans(n_clusters=4, init="k-means++", n_init=10, random_state=0),
)
labels = model.fit_predict(logs[["GR", "RHOB", "NPHI", "LOG_RT"]])
km = model[-1]
centres = model[0].inverse_transform(km.cluster_centers_)  # original units
print(km.inertia_, km.n_iter_)
```

Since scikit-learn 1.4, n_init="auto" runs k-means++ once: set n_init yourself.

Read the code line by line. The pipeline keeps the scaler and the model together, so the same fitted scaling is applied whenever the model predicts. The highlighted line fixes the three settings that make the result reproducible: k-means++ seeding, ten initializations (keep the lowest inertia), and a fixed random_state.

logs is a DataFrame with a LOG_RT column already computed as log10 of resistivity. fit_predict returns one label per row. model[-1] is the fitted KMeans step; model[0] is the fitted scaler, whose inverse_transform turns centroids back into original units (API, g/cm³, v/v, log10 ohm·m) for profiling.

inertia_ is the final objective J; n_iter_ is the number of Lloyd iterations of the best run.

For new wells, call model.predict on the same columns in the same units; do not refit the scaler. Save the fitted pipeline with joblib together with a note of the feature list and units. [2 min]

How many restarts are enough?

PRACTICE

5 min Pairs

How many restarts are enough?

In the random-start experiment, 5 of 20 starts reached the lowest inertia.

1. Estimate p, the chance that one random start succeeds.
2. What is the chance that n_init = 10 finds the best solution at least once?
3. How many starts give 99%, and what are you assuming?

Machine Learning / Module 3 · Clustering and mixture models 30

Give three minutes, then solve together. [5 min]

Worked answer. $p \approx 5/20 = 0.25$. With n independent starts the chance that none succeeds is $(1 - p)^n$, so the chance of at least one success with n = 10 is $1 - 0.75^{10} = 1 - 0.056 = 0.944$.

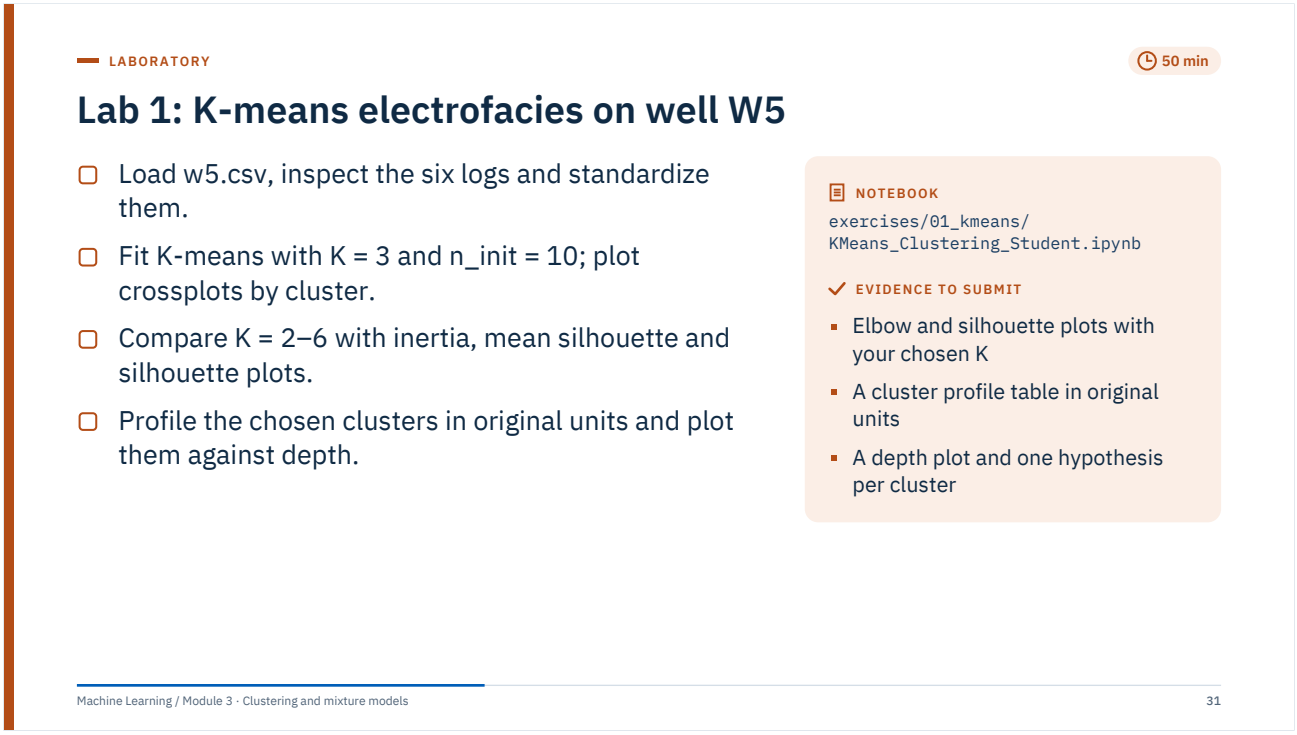
For 99% we need $0.75^n < 0.01$. $0.75^{16} = 0.0100$ (just above), $0.75^{17} = 0.0075$, so n = 17 starts.

Assumptions: starts are independent and each has the same success probability; p is estimated from only 20 runs, so it is uncertain (5 successes in 20 is compatible with a p anywhere from roughly 0.1 to 0.5).

Point out the practical lesson: better seeding changes p more than extra restarts do. On the same data a single k-means++ start succeeded in 98.5% of 200 runs, so n_init = 10 with k-means++ is very safe here, while n_init = 10 with uniform random starts would fail about one time in eighteen.

Common error: computing $10 \times 0.25 = 2.5$ and calling it a probability.

Lab 1: K-means electrofacies on well W5



Timing: 40 minutes of work and a 10-minute debrief. [50 min]

Setup: the notebook is in exercises/01_kmeans. w5.csv has 7,071 samples of DEPTH, Pressure, AI (acoustic impedance), VpVs, SW, phid and vcl with no missing values. The pairplot and the centroid heatmap use seaborn, which requirements-labs.txt installs. The notebook labels its scaling as exploratory: the scaler is fitted on the whole well because nothing is predicted for other wells.

Walk the room and check three things. First, that DEPTH is excluded from the features. Second, that learners look at the SW histogram: 84% of samples have SW exactly 1.0, so SW behaves almost like a flag. Third, that Pressure is examined against depth: their correlation is 0.96, so including Pressure puts depth into the features.

Expected results (n_init = 10, random_state = 42): mean silhouette 0.37, 0.41, 0.32, 0.32 and 0.31 for K = 2 to 6, so the notebook's rule (highest mean silhouette, checked against the elbow) picks K = 3. Values around 0.3 to 0.4 mean weakly separated groups. The depth section then prints each feature's correlation with DEPTH (Pressure 0.96, AI 0.72) and the depth range of each cluster, as evidence for one hypothesis per cluster.

Common errors: forgetting to exclude DEPTH; plotting centroids in standardized units and reading them as physical values; treating the best silhouette as proof of three facies.

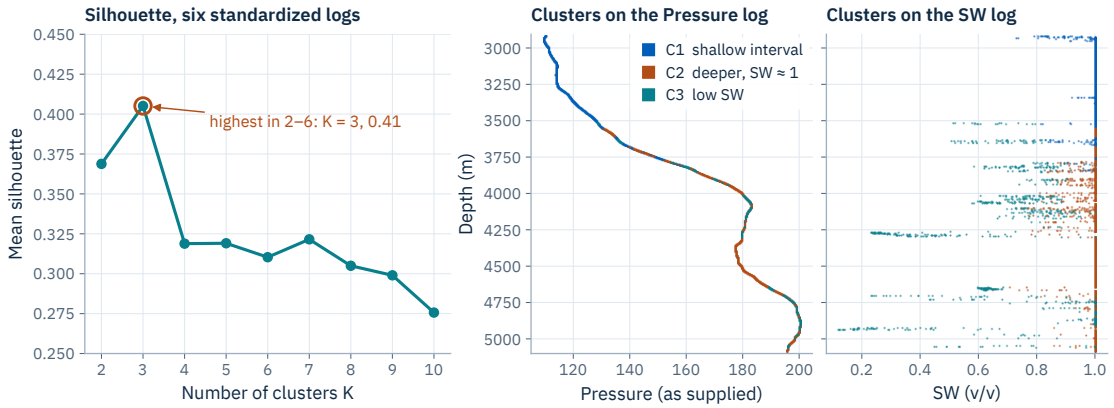
Fast finishers: rerun without Pressure and compare the labels with adjusted_rand_score.

Lab 1 debrief: W5 clusters follow depth and SW

DISCUSSION

Lab 1 debrief: W5 clusters follow depth and SW

The best silhouette (0.41, K = 3) mostly splits depth and low SW, not lithology.



Computed from w5.csv (course data): six standardized logs, KMeans(n_init = 10, random_state = 42), as in the notebook.

Machine Learning / Module 3 · Clustering and mixture models 32

Show the reference result and ask groups to compare their own. [10 min]

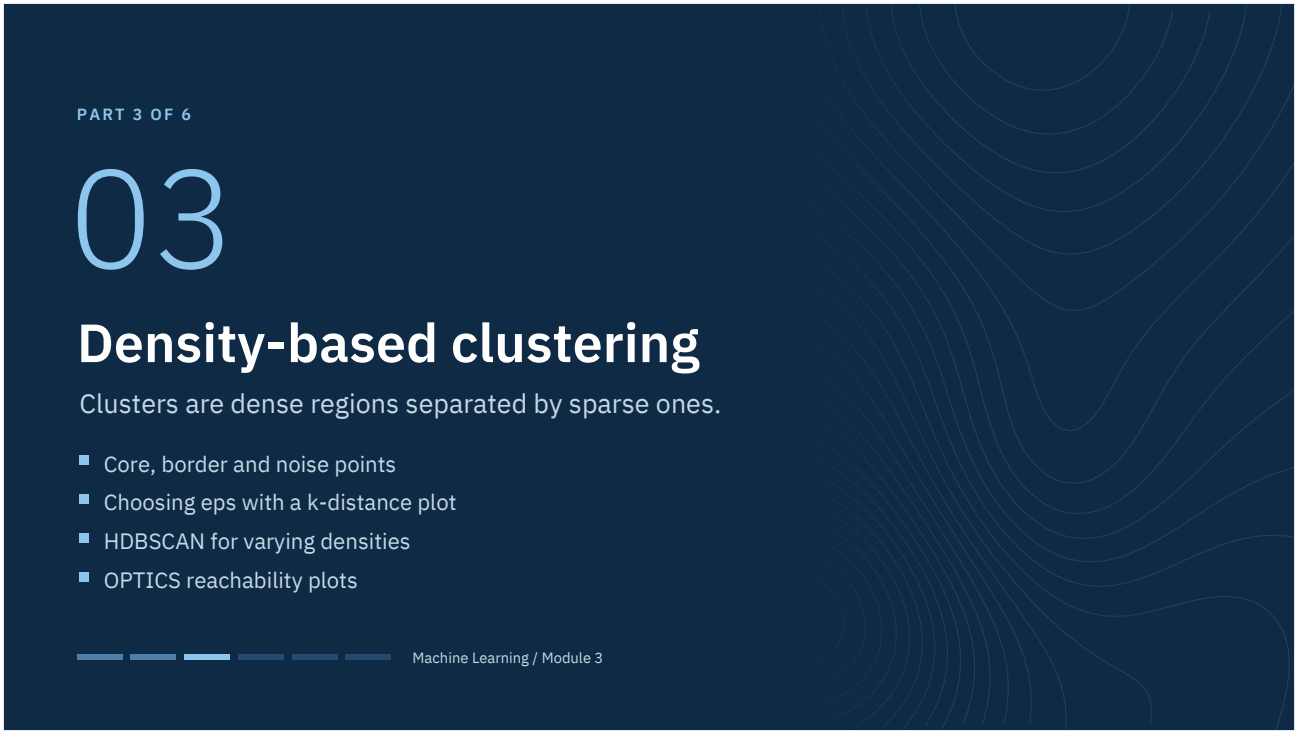
Left: mean silhouette peaks at K = 3 (0.41) but every value is between 0.28 and 0.41: the data are not strongly clustered.

Middle and right: the three clusters against depth. C1 (2,622 samples, 88% above 3,700 m) is essentially the shallow interval where Pressure is low. C3 (833 samples) collects the samples with SW below 1 (mean SW 0.70). C2 (3,616 samples) is the deeper water-bearing interval. So the "electrofacies" are largely a depth split plus a fluid flag.

Discuss why. Pressure increases with depth (correlation 0.96), and AI and VpVs also carry a compaction trend with depth. Without Pressure the K = 3 labels still agree fairly well with the six-feature result (adjusted Rand index 0.74), so the depth trend is carried by several logs.

Lesson: clusters followed the dominant variance, which here is depth and fluid, not lithology. To target lithology, remove or detrend depth-related variables, drop or rethink SW, and check against core or cuttings. Units of Pressure are not documented in the file, so describe it as supplied.

Density-based clustering



Density methods answer a different question from K-means: where are the dense regions, and which observations belong to none of them? They do not need K, they can follow curved shapes, and they label sparse observations as noise.

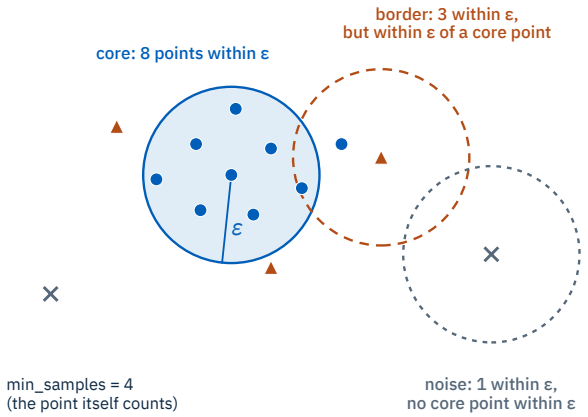
They are not upgrades that dominate K-means. They need a density scale, they struggle in high dimensions where distances concentrate, and in scikit-learn they cannot assign new samples without refitting. By the end of the part learners should be able to choose eps from a k-distance plot, explain a noise fraction, and know when HDBSCAN is the better starting point. [1 min]

DBSCAN labels points as core, border or noise

DENSITY-BASED CLUSTERING

DBSCAN labels points as core, border or noise

- **Core**: at least min_samples points within ε, counting itself
- Border: not core, but within ε of a core point
- Noise: neither; label −1
- Clusters are core points linked through overlapping ε-neighbourhoods



Hand-placed points; types computed with DBSCAN(eps = 1.0, min_samples = 4).

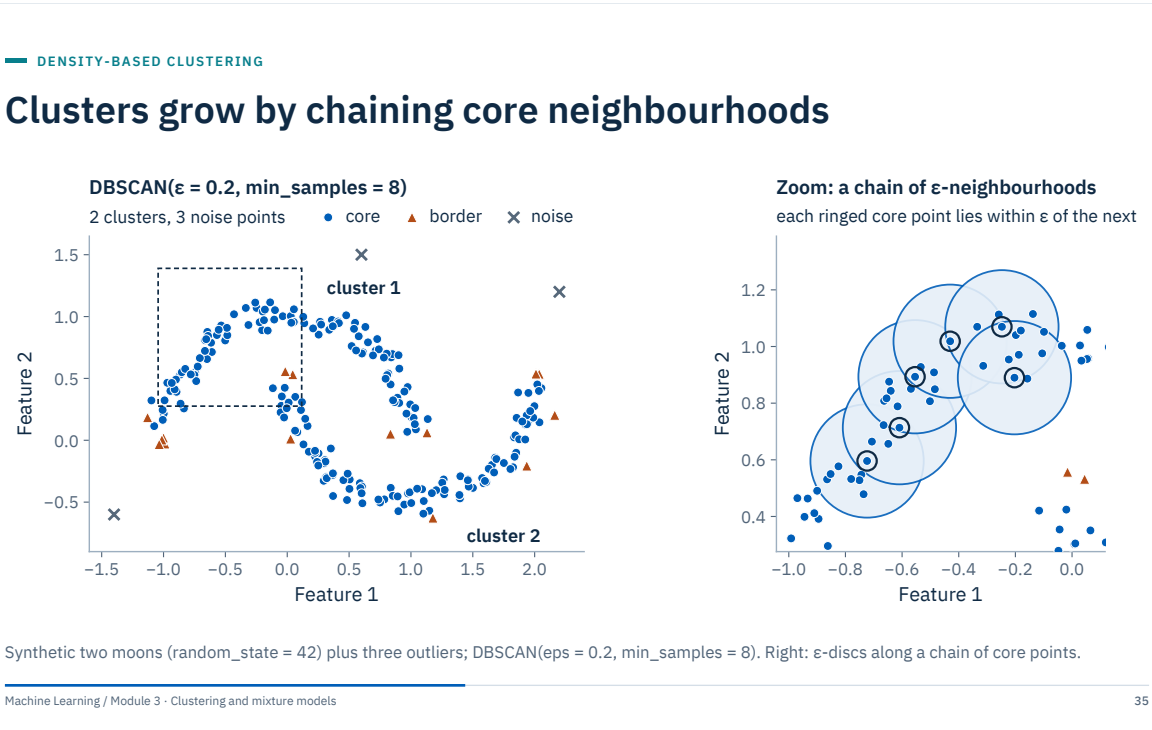
Machine Learning / Module 3 · Clustering and mixture models 34

Walk through the three circles. The blue point has 8 points within ε, including itself, so with min_samples = 4 it is a core point. The orange triangle has only 3 points within ε, so it is not core, but one of those is a core point, which makes it a border point of the same cluster. The grey cross on the right has only itself within ε and no core point nearby, so it is noise.

Two scikit-learn details to state explicitly. min_samples counts the point itself. A border point within ε of core points from two clusters is assigned to whichever cluster reaches it first, so border assignments can depend on data order; core points and noise do not.

Noise is an algorithmic label at the chosen settings, not proof of a faulty measurement. In a well log it may be a thin bed, a washout or a rare lithology. The method comes from Ester and colleagues (1996). [3 min]

Clusters grow by chaining core neighbourhoods



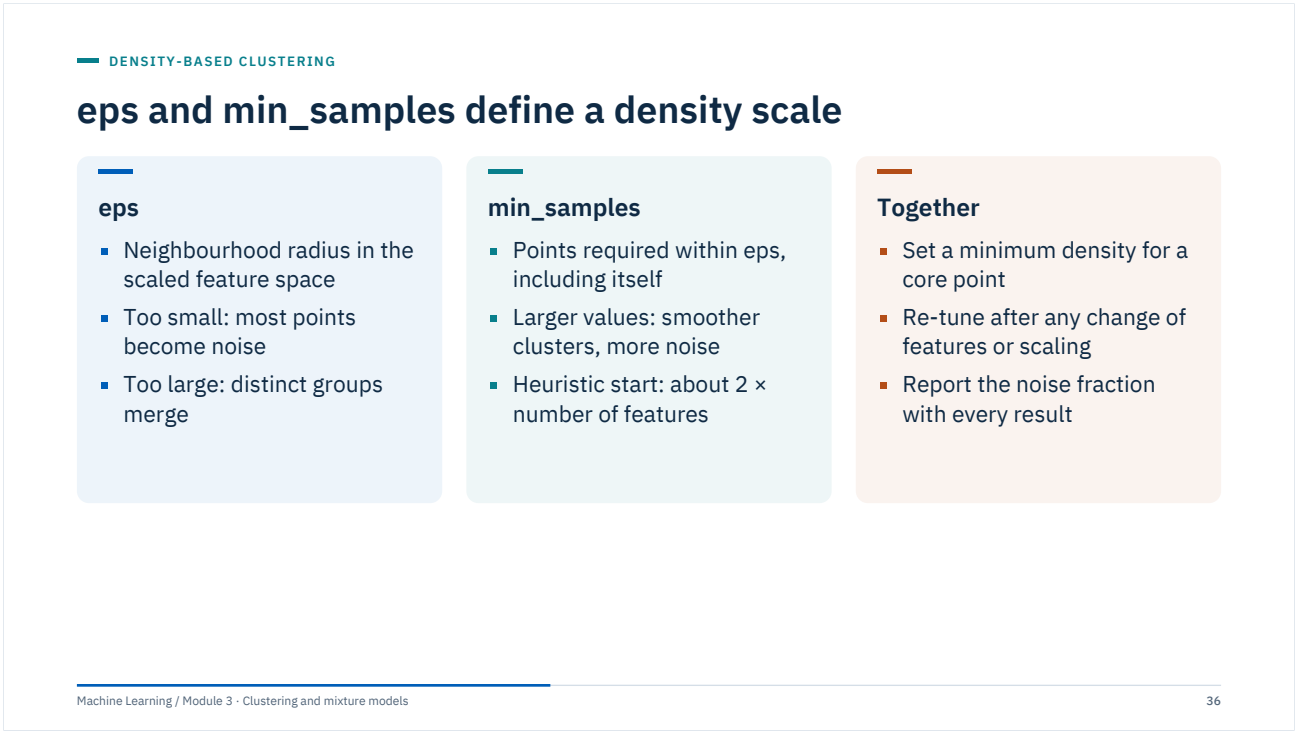
This is DBSCAN on the two-moon data with three added outliers (eps = 0.2, min_samples = 8): two clusters and three noise points. Blue circles are core points, orange triangles border points and grey crosses noise. The right panel zooms into the dashed box: each ringed core point lies within ε of the next, so their ε-discs overlap and the cluster keeps growing along the chain.

The mechanism: start at any unvisited core point, add all points within ε, and keep expanding through every core point reached. The cluster is everything density-reachable from its core points, so it can bend around a curve as long as the chain of core neighbourhoods is unbroken.

Point out that border points sit on the thin edges of each moon, and that the three isolated points became noise. Also note what could go wrong: a line of stray points between the moons at the same density would chain them into one cluster, which is the same weakness single linkage has in Part 4.

Question: why are the tips of the moons often border rather than core points? Answer: at the tips there are fewer neighbours within ε, so the count falls below min_samples. [2 min]

eps and min_samples define a density scale



The two parameters together define a threshold density: at least min_samples points inside a ball of radius eps. Any point in a denser region is core.

eps is measured in the same units as the scaled features, so it must be re-tuned whenever features, transforms or scaling change. In the Lab 2 data the median 5-distance after standardization is 0.37 for two logs and 0.27 for six logs, but only a k-distance plot on your own data can say what fits.

min_samples controls how much support a core point needs. Larger values smooth out small, noisy clusters and increase the noise fraction. A common heuristic starting point in the density-clustering literature is twice the number of features; treat it as a starting point, not a rule.

Always report the noise fraction next to the number of clusters: "3 clusters with 40% noise" and "3 clusters with 2% noise" are very different results.

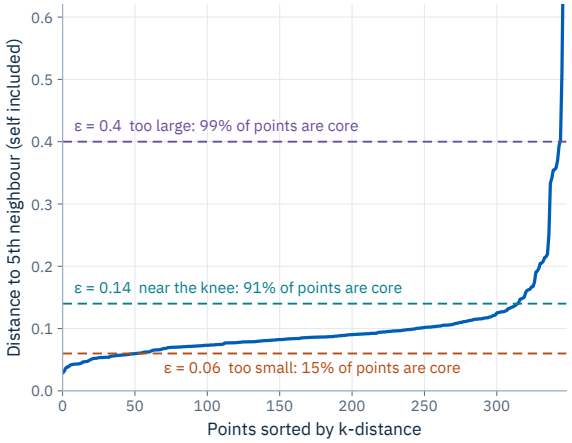
Transition: the k-distance plot turns this into a visual choice. [2 min]

A k-distance plot suggests a range for eps

DENSITY-BASED CLUSTERING

A k-distance plot suggests a range for eps

- For each point, find the distance to its min_samples-th neighbour
- Sort and plot these k-distances
- Points below the chosen eps are core points
- Pick eps near the knee, then check a range around it



Synthetic moons (random_state = 3) plus 28 uniform noise points; distances to the 5th neighbour, counting the point itself.

Machine Learning / Module 3 · Clustering and mixture models 37

Compute, for every point, the distance to its k-th nearest neighbour with k = min_samples, counting the point itself as scikit-learn's DBSCAN does. With NearestNeighbors(n_neighbors=min_samples).fit(Z).kneighbors(Z), the first neighbour returned is the point itself, so the last column is exactly the right distance.

Sort and plot. Points in dense regions have small k-distances (the long flat part); isolated points have large ones (the steep tail). Any point whose k-distance is below eps is a core point, so the curve directly tells you the fraction of core points at each eps.

Read the three lines. At ε = 0.06 only 15% of points are core; at 0.14, near the knee, 91%; at 0.4, 99%, including many of the scattered noise points.

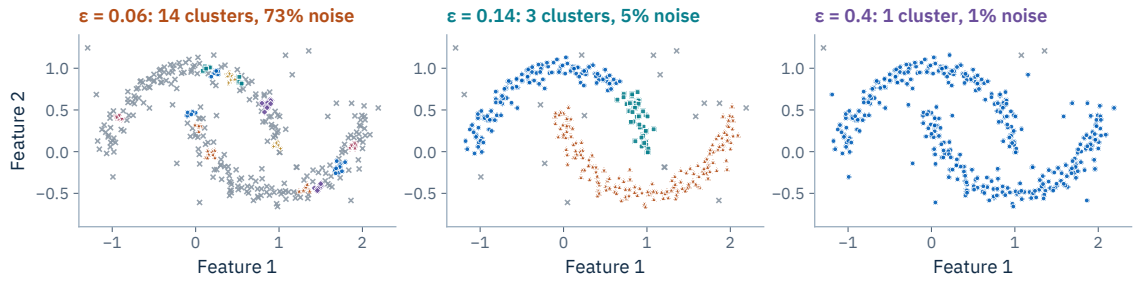
The knee is a suggestion, not an answer: in real data there may be no sharp knee, or several. Try a range and look at the clusters, which the next slide does. [3 min]

Small changes in eps change the answer

DENSITY-BASED CLUSTERING

Small changes in eps change the answer

The knee value recovers both moons; either side of it DBSCAN fragments or merges them.



Same synthetic data as the k-distance plot; DBSCAN(min_samples = 5) at three eps values. Grey crosses are noise.

Machine Learning / Module 3 · Clustering and mixture models 38

Left, ε = 0.06: most points are noise (73%) and the remaining core points form 14 small fragments. Middle, ε = 0.14: the two moons are recovered, with 5% noise, but one moon's tip has split into a third small cluster, which shows that even the knee value is not perfect. Right, ε = 0.4: everything is one cluster and only 1% is noise, because the neighbourhoods now bridge the gap between the moons.

The lesson for practice is a sensitivity table: run a small grid of eps and min_samples values and record the number of clusters, the noise fraction and the size of the largest cluster. Stable conclusions should hold across a range of settings, not at one magic value.

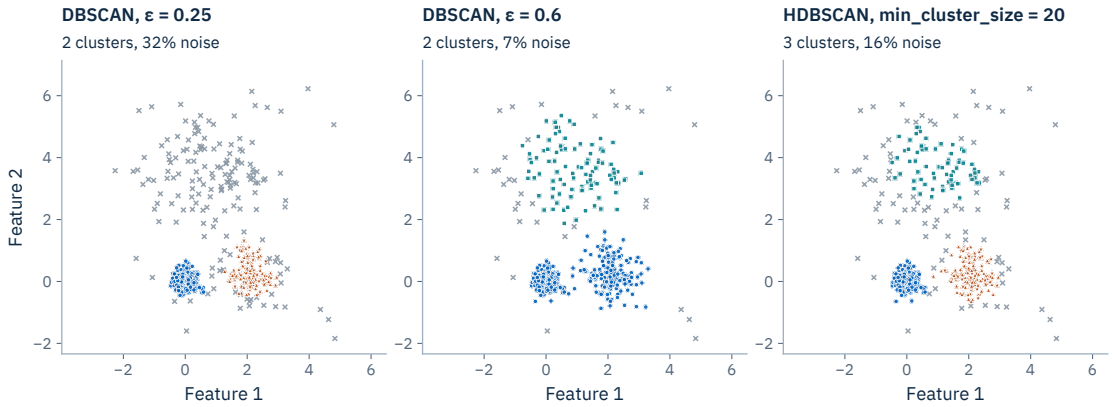
This is exactly Lab 2, step 6. [2 min]

One eps cannot fit clusters of different density

DENSITY-BASED CLUSTERING

One eps cannot fit clusters of different density

Small eps loses the sparse group; large eps merges the dense ones. HDBSCAN adapts.



Synthetic dense, medium and sparse groups plus uniform background (seed 14); DBSCAN(min_samples = 10) and HDBSCAN(min_cluster_size = 20).

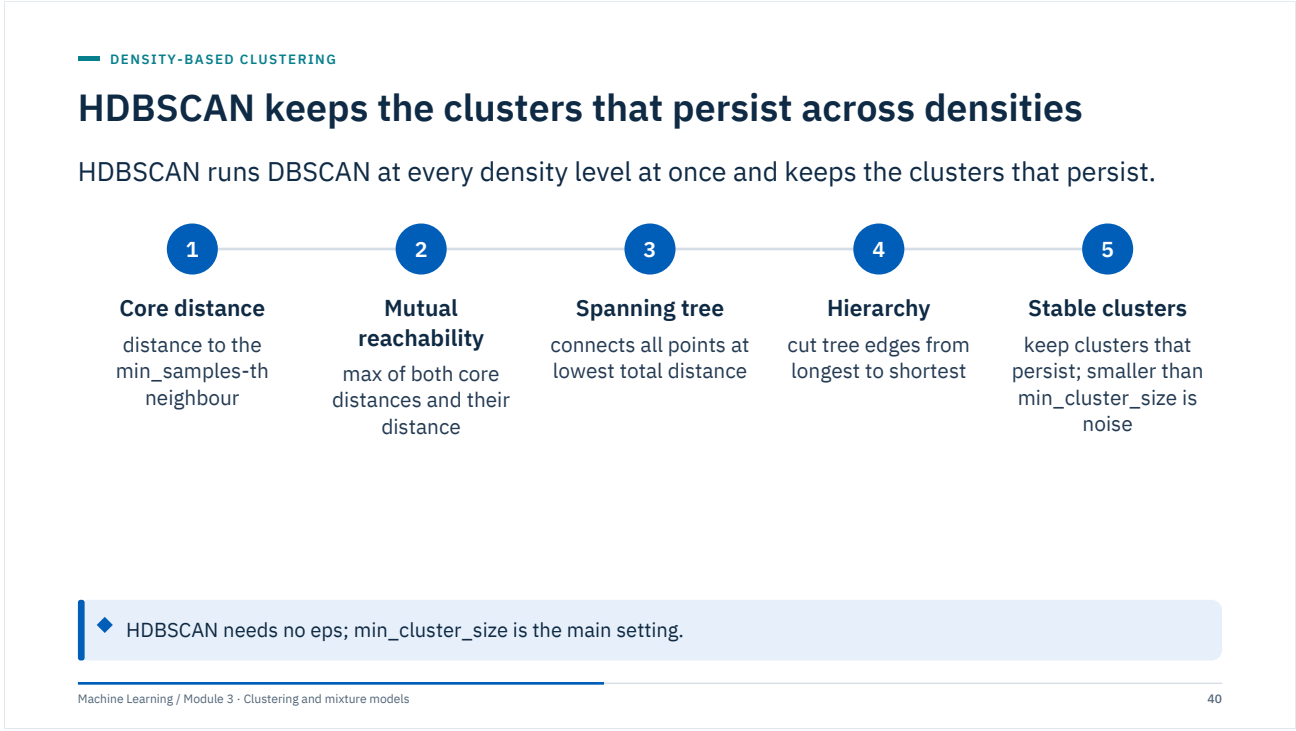
The data contain three groups with very different spreads plus a scattered background.

With ε = 0.25, DBSCAN finds the dense and medium groups but calls the whole sparse group noise (32% noise). With ε = 0.6 it recovers the sparse group but merges the dense and medium groups into one cluster (2 clusters, 7% noise). No single ε gets all three right, because DBSCAN applies one global density threshold.

HDBSCAN (scikit-learn 1.3 and later) finds all three groups with one intuitive parameter, the minimum cluster size, and labels 16% as noise, mostly the background and the fringe of the sparse group. It also returns a membership strength per point (probabilities_).

This matters for logs and sensor data, where groups often differ in spread: tight carbonate responses next to broad shale responses, or a steady operating state next to a variable one. [3 min]

HDBSCAN keeps the clusters that persist across densities



Explain the pipeline without the mathematics. Step 1: each point's core distance measures how sparse its neighbourhood is. Step 2: the mutual reachability distance between two points is the largest of their two core distances and their actual distance, which pushes sparse points away from everything while leaving dense regions unchanged. Step 3: a minimum spanning tree connects all points using these distances. Step 4: removing edges from the longest to the shortest gives a hierarchy of clusters, the same as running DBSCAN (without border points) at every possible eps. Step 5: clusters that persist over a wide range of density levels are kept; groups smaller than min_cluster_size are treated as points falling out of a cluster.

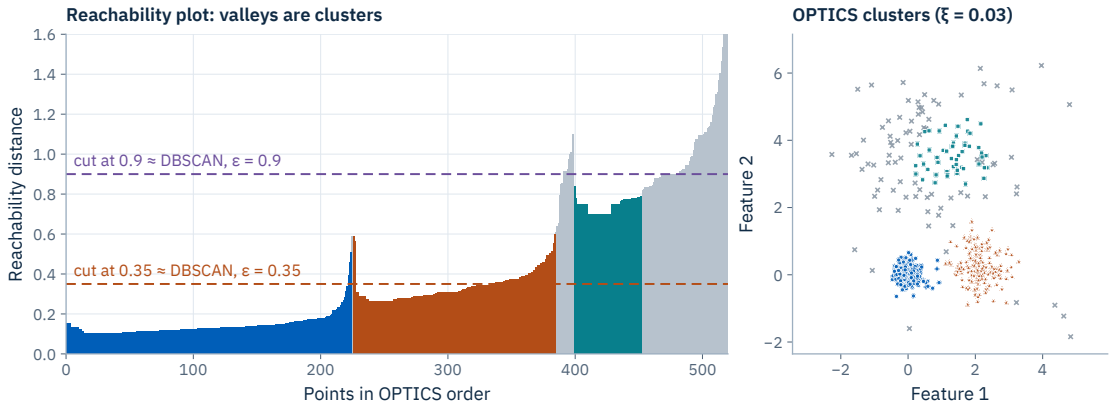
In scikit-learn, min_samples defaults to min_cluster_size. cluster_selection_method="leaf" gives finer clusters than the default "eom". The algorithm is from Campello, Moulavi and Sander (2013). [3 min]

OPTICS orders points by reachability

DENSITY-BASED CLUSTERING

OPTICS orders points by reachability

Valleys are dense regions; a cut at height ε reproduces DBSCAN with that ε.



Same synthetic data as the HDBSCAN comparison; OPTICS(min_samples = 30, xi = 0.03). Bars are coloured by extracted cluster; grey = noise.

Machine Learning / Module 3 · Clustering and mixture models 41

OPTICS (Ankerst and colleagues, 1999) visits points in an order that keeps dense regions together and records a reachability distance for each point. Plotted in that order, dense clusters appear as valleys and sparse regions as peaks.

Read the left panel: the deep blue valley is the dense group, the orange valley the medium group, and the shallow teal valley the sparse group. Grey bars are noise.

A horizontal cut at any height gives the DBSCAN clustering at that eps. Cutting at 0.35 reproduces DBSCAN with the same min_samples exactly on these data; the cut at 0.9 almost exactly. The ξ method instead extracts valleys by the steepness of their walls, so clusters of different density can be extracted together, as in the right panel.

Keep this brief: it is an alternative view of the same idea as HDBSCAN. Its reachability plot is a useful diagnostic even when the final model is DBSCAN. [2 min]

HDBSCAN and OPTICS adapt where DBSCAN cannot

DENSITY-BASED CLUSTERING

HDBSCAN and OPTICS adapt where DBSCAN cannot

Method	Main settings	Varying density	New samples
DBSCAN	eps, min_samples	Poor: one global eps	No predict: refit
HDBSCAN	min_cluster_size	Good	No predict in scikit-learn
OPTICS	min_samples, xi	Good	No predict: refit
K-means (reference)	K	Poor	predict: nearest centroid

All three density methods label noise as –1 and need a scaled, meaningful distance.

Summarize the density family. DBSCAN is simple and fast, but one global eps limits it when densities differ. HDBSCAN and OPTICS handle varying densities.

A practical difference from K-means and Gaussian mixtures: none of the three density estimators in scikit-learn has a predict method. To label new wells you either refit on the combined data (and then labels may change) or train a classifier, for example k-nearest neighbours, on the clustered samples and their labels. The separate hdbscan package offers an approximate prediction function, but that is outside scikit-learn.

All three rely on a meaningful distance, so everything from Part 1 about scaling and feature choice applies with even more force: the density threshold is measured in scaled units. As the number of features grows, distances concentrate and density contrasts fade; consider reducing dimensionality first (Module 4). [2 min]

Tune eps from k-distances, then report the noise

DENSITY-BASED CLUSTERING

Tune eps from k-distances, then report the noise

```
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN

ms = 5                                          # counts the point itself
nn = NearestNeighbors(n_neighbors=ms).fit(Z)    # Z: scaled features
kdist = np.sort(nn.kneighbors(Z)[0][:, -1])     # plot; look for a knee

db = DBSCAN(eps=0.5, min_samples=ms).fit(Z)
labels = db.labels_                             # -1 marks noise
n_clusters = len(set(labels) - {-1})
noise_frac = np.mean(labels == -1)
print(n_clusters, round(noise_frac, 3))
```

Report the number of clusters, the noise fraction and the settings together.

Machine Learning / Module 3 · Clustering and mixture models 43

Z is the standardized feature array. The highlighted lines are the two decisions: k (equal to min_samples, counting the point itself) for the k-distance plot, and eps chosen from that plot.

nn.kneighbors(Z) returns two arrays, distances and indices; [0] takes the distances and [:, -1] the distance to the ms-th neighbour, where the first neighbour is the point itself. Sort and plot kdist to find the knee.

labels_ contains −1 for noise. n_clusters excludes the noise label. core_sample_indices_ lists the core points if you want to separate core from border.

For the six W5 logs in Lab 2 the median 5-distance after standardization is 0.27 and the 90th percentile 0.52, so at eps = 0.5 about 89% of samples are core points and 6.6% end up as noise. Swap DBSCAN for HDBSCAN(min_cluster_size=...) with the same labels_ convention when densities differ. [2 min]

Density clustering diagnosis

PRACTICE

5 min Pairs

Density clustering diagnosis

DBSCAN assigns 85% of observations to noise.

1. List two possible explanations other than bad data.
2. What would you plot or inspect before changing eps?
3. What happens if you simply raise eps until noise is below 5%?

Give three minutes, then collect answers. [5 min]

Worked answer. Explanations other than bad data: eps too small for the scaled units; features not scaled, so one feature's large numeric range dominates distances; an unsuitable distance metric; too many features, so distances concentrate and every neighbourhood looks sparse; sampling too sparse for the chosen min_samples; min_samples too large; or genuinely diffuse data without dense groups.

Inspect before changing eps: the k-distance plot for the current min_samples; the feature distributions after scaling; the noise fraction by well, depth interval or time period (is noise concentrated in one well or tool run?); and a two-dimensional view of where noise falls.

Raising eps until noise is below 5% usually merges distinct groups into one large cluster, as at $\varepsilon = 0.4$ on the moons. Noise fraction is not a quantity to minimize; it is a consequence of the density scale you chose.

Common error: tuning eps on raw, unscaled data and concluding that the method does not work.

Lab 2: density clustering on log data

LABORATORY

45 min

Lab 2: density clustering on log data

- ☐ Standardize SP and GR from log.csv; run DBSCAN with eps = 0.5, min_samples = 5.
- ☐ Draw the k-distance plot and mark the default eps on it.
- ☐ Sweep eps and min_samples; tabulate clusters, noise and largest cluster.
- ☐ Repeat on the six W5 logs and inspect what the noise points have in common.

NOTEBOOK

`exercises/02_dbscan/DBSCAN_Clustering_Student.ipynb`

EVIDENCE TO SUBMIT

- A k-distance plot with your chosen eps
- A sensitivity table for eps and min_samples
- Two sentences on what the noise represents

Machine Learning / Module 3 · Clustering and mixture models 45

Timing: 35 minutes of work and a 10-minute debrief. [45 min]

Setup: log.csv has 110 samples of SP and GR; w5.csv is the Lab 1 file. As in Lab 1, each scaler is fitted on the whole file, and the notebook labels this as exploratory.

Expected results on log.csv (standardized, min_samples = 5): eps = 0.5 gives 1 cluster and 8 noise points. The median 5-distance is 0.37 and the 90th percentile 0.56, so 0.5 sits just above most k-distances. Sweeping eps: 0.2 gives 105 noise points; 0.3 gives 5 small clusters and 57 noise; 0.8 and above give one cluster with 1 to 4 noise points.

The sweep table also reports the largest cluster, and every silhouette in the notebook leaves out the noise label. Section 7 compares with K-means: DBSCAN finds one cluster, so the notebook fits K = 2 and reports the DBSCAN silhouette as undefined. For W5 (Section 8) eps is read from the k-distance plot: the median 5-distance is 0.27 and the 90th percentile 0.52, so the notebook uses eps = 0.5 (89% of samples are core points) and tabulates eps = 0.3 to 1.0, reproducing the debrief table. Section 9 then compares the noise points with the other samples in original units.

Common errors: forgetting to exclude label −1 when counting clusters; using NearestNeighbors with a different k from min_samples; tuning eps on unscaled data.

Fast finishers: try HDBSCAN(min_cluster_size=10) on the W5 logs and compare the noise points.

Lab 2 debrief: noise flags rare responses, not errors

DISCUSSION

Lab 2 debrief: noise flags rare responses, not errors

eps (standardized)	log.csv: clusters, noise	W5: clusters, noise	W5 largest cluster
0.3	5, 57 of 110	76, 30%	1,699
0.5	1, 8 of 110	15, 6.6%	6,342
0.8	1, 4 of 110	3, 1.2%	6,969
1.0	1, 1 of 110	3, 0.3%	7,037

With min_samples = 5, both datasets are essentially one dense region plus sparse outliers.

Show the reference table and ask groups to compare their own numbers. [10 min]

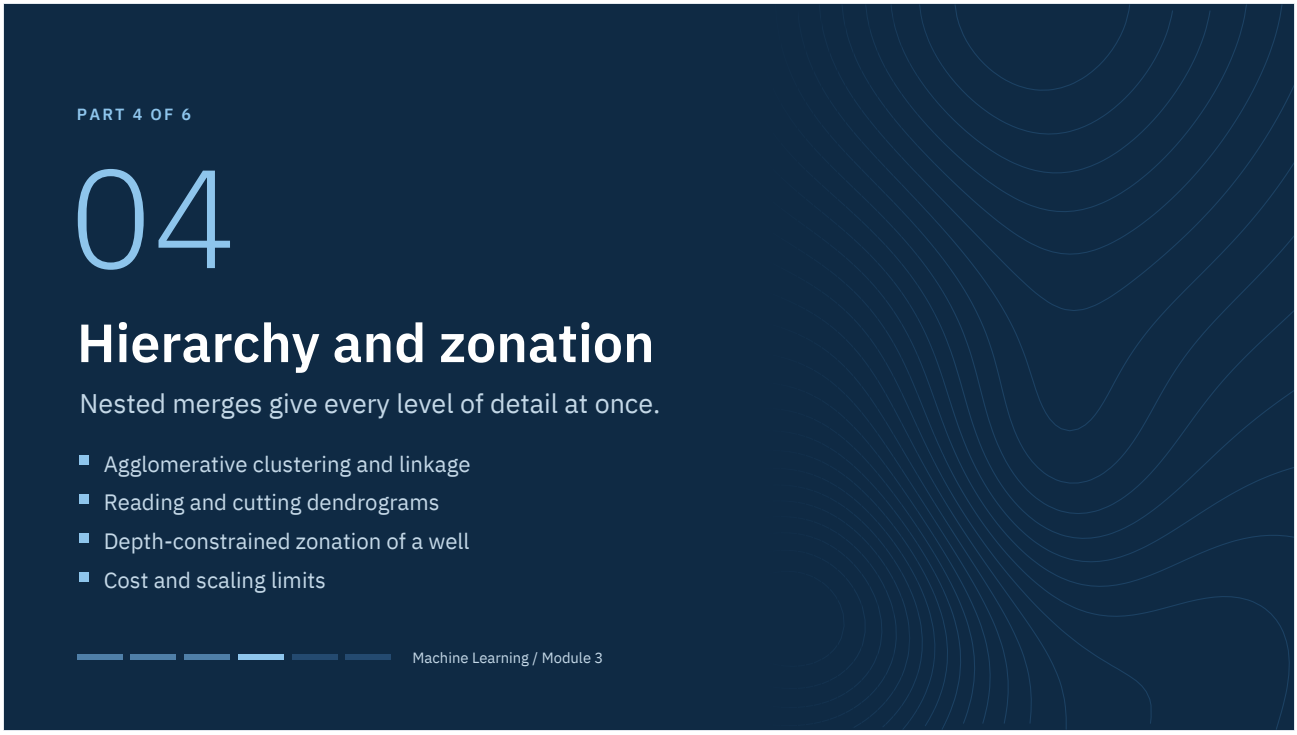
log.csv: at every eps from 0.5 upwards DBSCAN finds one cluster plus a few noise points; smaller eps values fragment the data. The SP–GR sample does not contain separate dense groups at this sampling density.

W5: the six-log data behave the same way. From eps = 0.5 upwards one cluster holds most of the 7,071 samples (90% at 0.5, 99% at 0.8), and at eps = 0.8 and 1.0 the other "clusters" contain fewer than ten samples each. So the three K-means clusters from Lab 1 were cuts through one continuous, connected cloud, which is consistent with their low silhouettes (0.3 to 0.4).

What did the noise points have in common? At eps = 0.5, 71% of the 469 noise points have SW below 1, against 12% of the other samples, and their clay volume is lower; they occur in short runs (median two samples, about 0.6 m). The notebook prints all three checks in Section 9. Before calling them anomalies, check them against depth: thin hydrocarbon-bearing beds, fluid contacts and tool effects are all candidates.

Key message: DBSCAN and K-means answered different questions, and both answers are informative.

Hierarchy and zonation



Hierarchical clustering produces a whole family of partitions, from every observation on its own to everything in one cluster. The number of clusters can be chosen after the computation by cutting the tree, and the tree itself shows which groups are close relatives.

For subsurface work its most useful form is constrained clustering: only neighbouring depth samples may merge, which turns clustering into automatic zonation of a log. That links directly to the log-zonation panel from the opening.

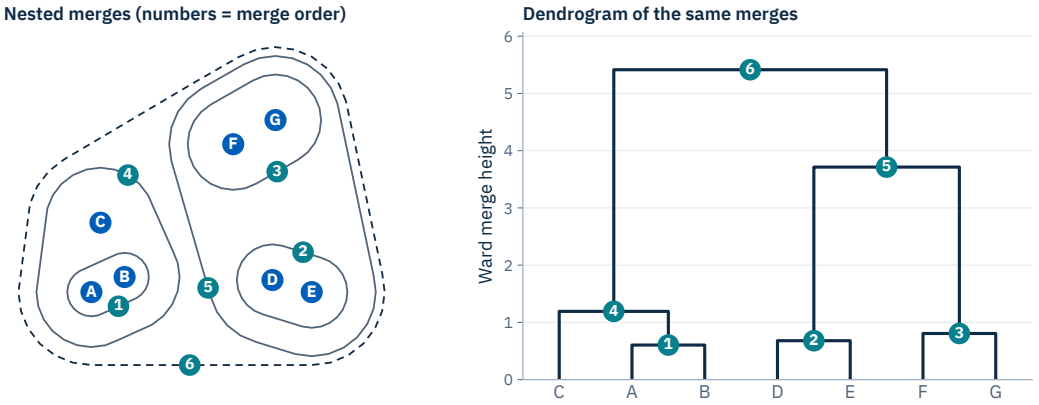
The cost is memory and time that grow at least with the square of the number of samples, which we quantify at the end of the part. [1 min]

Agglomerative clustering builds a merge history

HIERARCHY AND ZONATION

Agglomerative clustering builds a merge history

Start with singletons, merge the closest pair of clusters, and record each height.



Seven hand-placed points; scipy Ward linkage. Merge numbers follow the rows of the linkage matrix.

Follow the numbers. Merge 1 joins A and B, the closest pair. Merges 2 and 3 join D with E and F with G. Merge 4 adds C to the A–B pair. Merge 5 joins the D–E and F–G pairs. Merge 6 joins everything.

On the right, the same merges appear as a dendrogram: each horizontal bar is a merge, and its height is the linkage distance at which it happened (here the Ward criterion). Heights never decrease going up for the standard linkages, so the tree can be cut at any height.

Two implementation notes. In scipy, linkage returns an (n − 1) × 4 matrix: the two clusters merged, the merge height and the new cluster size. In scikit-learn, AgglomerativeClustering returns labels for a chosen n_clusters or distance_threshold.

Ask: after merge 4, how many clusters are there? Answer: three (A–B–C, D–E, F–G). [3 min]

Linkage changes the meaning of close

HIERARCHY AND ZONATION

Linkage changes the meaning of close

Linkage	Distance between groups	Common behaviour
Single	Closest pair of observations	Can chain through bridges
Complete	Farthest pair	Favours compact groups
Average	Mean of all pairwise distances	Intermediate behaviour
Ward	Increase in within-cluster squared error	Compact, similar-size groups

Ward is defined for Euclidean distance; the other three accept any distance.

Machine Learning / Module 3 · Clustering and mixture models 49

The linkage decides which two clusters are "closest" at each step.

Single linkage uses the closest pair of observations, so two groups merge as soon as any two members are close; a thin bridge of points is enough. Complete linkage uses the farthest pair, so a cluster merges only when all its members are close to the other cluster, which favours compact groups. Average linkage uses the mean of all pairwise distances.

Ward is a variance criterion, not a pairwise distance formula: it merges the pair of clusters whose union increases the total within-cluster sum of squares the least, the same objective as K-means. It tends to give compact clusters of similar size. Its usual implementation requires Euclidean distance; in scikit-learn metric must be "euclidean" when linkage="ward".

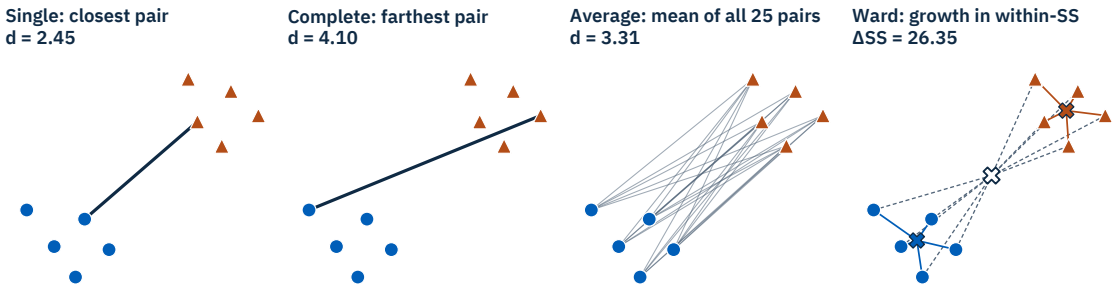
Question: which linkage is most vulnerable to a thin chain joining two dense regions? Answer: single linkage. [2 min]

Four linkages measure the same two groups differently

HIERARCHY AND ZONATION

Four linkages measure the same two groups differently

Same two groups, four definitions of the distance between them.



Two hand-placed five-point groups. Ward: increase in within-cluster sum of squares if the groups merge (dashed = distances to the merged centroid).

Machine Learning / Module 3 · Clustering and mixture models 50

The same two groups, four definitions of the distance between them.

Single: the closest pair, 2.45. Complete: the farthest pair, 4.10. Average: the mean of all 5 × 5 = 25 pairwise distances, 3.31. Ward: not a distance between points at all, but the growth in the within-cluster sum of squares if the groups merge, 26.35 in squared units. For two groups it equals nA·nB / (nA + nB) times the squared distance between their centroids: here 5 × 5 / 10 = 2.5 times the squared centroid distance.

Because the values differ, so does the merge order across a whole dataset, and therefore the tree. Single linkage looks only at the nearest edge; complete linkage is dominated by the most distant members, which makes it sensitive to outliers at the far side.

Transition: compute merge heights by hand for single and complete linkage. [2 min]

Merge heights by hand

PRACTICE

5 min Individual

Merge heights by hand

Five observations on a line: 0, 1, 3, 7, 8.

1. List the merges and their heights with single linkage.
2. Repeat with complete linkage.
3. Where would you cut each tree to get two clusters?

Give four minutes. [5 min]

Worked answer, single linkage. The closest pairs are 0–1 and 7–8, both at height 1 (either order). Next, 3 joins {0, 1} at height 2, the distance from 3 to 1. Finally {0, 1, 3} joins {7, 8} at height 4, the distance from 3 to 7. Heights: 1, 1, 2, 4.

Complete linkage. 0–1 and 7–8 at height 1. Then 3 joins {0, 1} at height 3, the distance from 3 to the farthest member, 0 (the alternative, 3 joining {7, 8}, would be at 5). Finally the two groups merge at 8, the distance from 0 to 8. Heights: 1, 1, 3, 8.

Two clusters: cut single linkage anywhere between 2 and 4, complete linkage between 3 and 8. Both give {0, 1, 3} and {7, 8}, but complete linkage shows a much clearer gap.

Check with scipy: linkage(np.array([[0],[1],[3],[7],[8]]), "single") gives heights 1, 1, 2, 4. Average linkage gives 1, 1, 2.5 and 6.17.

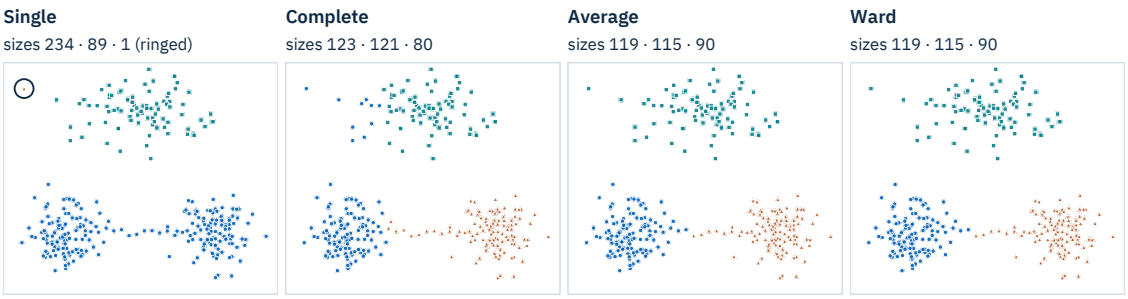
Common error: using centroid distances for complete linkage.

Single linkage chains through bridges

HIERARCHY AND ZONATION

Single linkage chains through bridges

Single linkage merges two blobs through a 14-point bridge and isolates one point.



Synthetic: two dense blobs joined by a thin bridge, plus an elongated third group (seed 5); AgglomerativeClustering(n_clusters = 3).

Same data, same number of clusters, four linkages.

Single linkage: the chain of bridge points links the two lower blobs, so they become one cluster of 234 points, the upper group is the second, and the third "cluster" is a single stray point at the far left (ringed). This chaining effect is the classic weakness of single linkage.

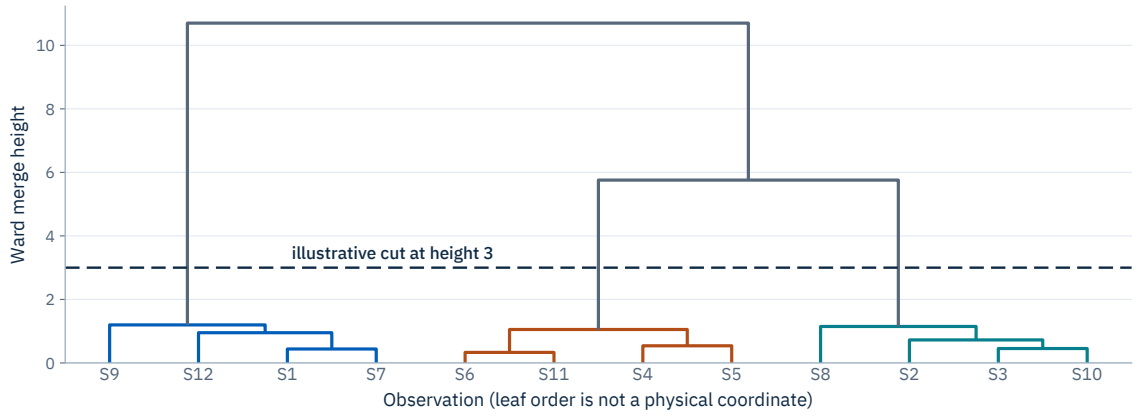
Complete, average and Ward separate the two blobs. They differ in where they cut the bridge and in how they handle the stray points of the upper group: complete linkage gives some of them to a lower blob.

Chaining is not always bad. For a long, continuous feature, such as a channel traced in a seismic attribute or a pipeline track in GPS data, following connected points is the goal. Choose the linkage for the shape you expect, as with the density methods. [2 min]

A dendrogram records merge heights

HIERARCHY AND ZONATION

A dendrogram records merge heights



Ward hierarchy of twelve synthetic observations (random_state = 42). Colours mark the three clusters below the dashed cut at height 3.

Machine Learning / Module 3 · Clustering and mixture models 53

Count the branches intersected by the dashed cut at height 3: three, so this cut gives three clusters.

Vertical merge height is informative: long vertical branches mean a cluster stayed separate over a wide range of heights, which is evidence of a real gap. The left-to-right arrangement of leaves is not: any branch can be rotated about its merge point without changing the hierarchy. So avoid claiming that adjacent leaves represent a measured spatial or depth ordering.

For large n, plot a truncated dendrogram (truncate_mode="lastp") showing only the last merges; the notebook does this with p = 30.

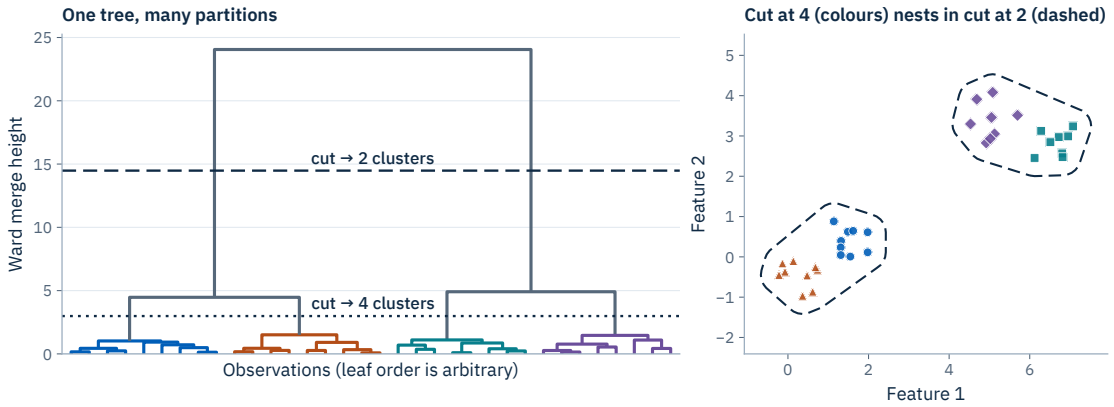
Ask: why is the merge height of the final merge so much larger than the others? Answer: Ward heights grow with the increase in within-cluster sum of squares (scipy reports the square root of twice that increase), which is large when well-separated groups are forced together. [2 min]

Cutting the tree at a height gives a partition

HIERARCHY AND ZONATION

Cutting the tree at a height gives a partition

Cuts at different heights give nested partitions of the same tree.



Synthetic 34 points from four blobs (random_state = 1); scipy Ward linkage; cuts halfway between merge heights; fcluster(criterion = "maxclust").

The dashed line cuts two vertical branches, giving two clusters; the dotted line cuts four. On the right, colours show the four-cluster partition and dashed outlines the two-cluster partition. Every four-cluster group sits entirely inside one two-cluster group: the partitions are nested, which K-means with K = 2 and K = 4 does not guarantee.

How to choose the cut: look for a large gap between successive merge heights. The last four merges happen at heights 1.5, 4.5, 4.9 and 24.1. The largest gap, from 4.9 to 24.1, supports two clusters; the next largest, from 1.5 to 4.5, supports four at a finer scale.

In scipy use fcluster(Z, t, criterion="distance") for a height or criterion="maxclust" for a number of clusters. In scikit-learn pass distance_threshold instead of n_clusters.

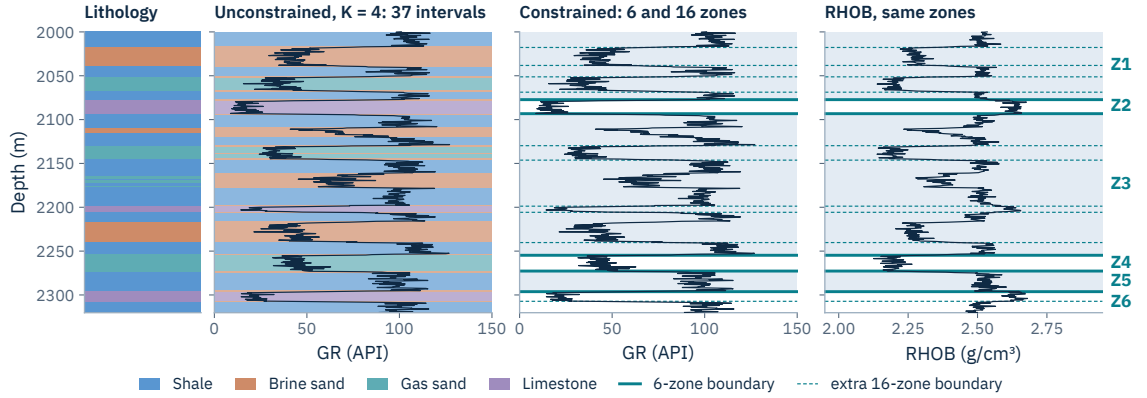
Transition: the most useful hierarchy for well data is one where only neighbouring samples can merge. [2 min]

Depth constraints turn clustering into zonation

HIERARCHY AND ZONATION

Depth constraints turn clustering into zonation

Merging only adjacent samples gives contiguous zones that nest across scales.



Synthetic well S-1 (seed 7): Ward on four standardized logs, unconstrained (K = 4) and depth-constrained.

Machine Learning / Module 3 · Clustering and mixture models 55

Left track: the simulated lithology. Second track: unconstrained Ward with K = 4 behaves like electrofacies clustering: samples are grouped by log response alone, producing 37 intervals. Most extra intervals are one- to four-sample slivers at bed boundaries, where tool averaging mixes two beds, and the whole thin-bedded unit near 2,165 m falls into the brine-sand cluster, because its averaged response lies between gas sand and shale.

Third and fourth tracks: the same Ward criterion, but a connectivity matrix only allows merges between adjacent depth samples. Every cluster is now a contiguous interval. The 6-zone cut (solid teal lines) places boundaries at major changes; the 16-zone cut (dashed) adds finer boundaries, always nested inside the coarse zones. The same boundaries mark the density track.

Zones and facies answer different questions: facies recur down the well, zones do not. Zone Z3 (about 2,094 to 2,255 m) contains shale, both sands, a limestone bed and the thin-bedded unit; for a coarse reservoir model that may be the right unit, and the 16-zone cut splits it where finer layering is needed.

Question: what should happen at a gap in the log, such as an unlogged interval? Answer: remove the adjacency across the gap, or the gap will be bridged. [3 min]

Zonation needs only a connectivity matrix

HIERARCHY AND ZONATION

Zonation needs only a connectivity matrix

```
from scipy.sparse import diags
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

logs = well.sort_values("DEPTH")
Z = StandardScaler().fit_transform(logs[["GR", "RHOB", "NPHI", "LOG_RT"]])
n = len(Z)
A = diags([np.ones(n - 1), np.ones(n - 1)], [-1, 1])  # adjacent samples
zones = AgglomerativeClustering(
    n_clusters=6, linkage="ward", connectivity=A
).fit_predict(Z)
```

Sort by depth first; remove links across logging gaps or well boundaries.

Machine Learning / Module 3 · Clustering and mixture models 56

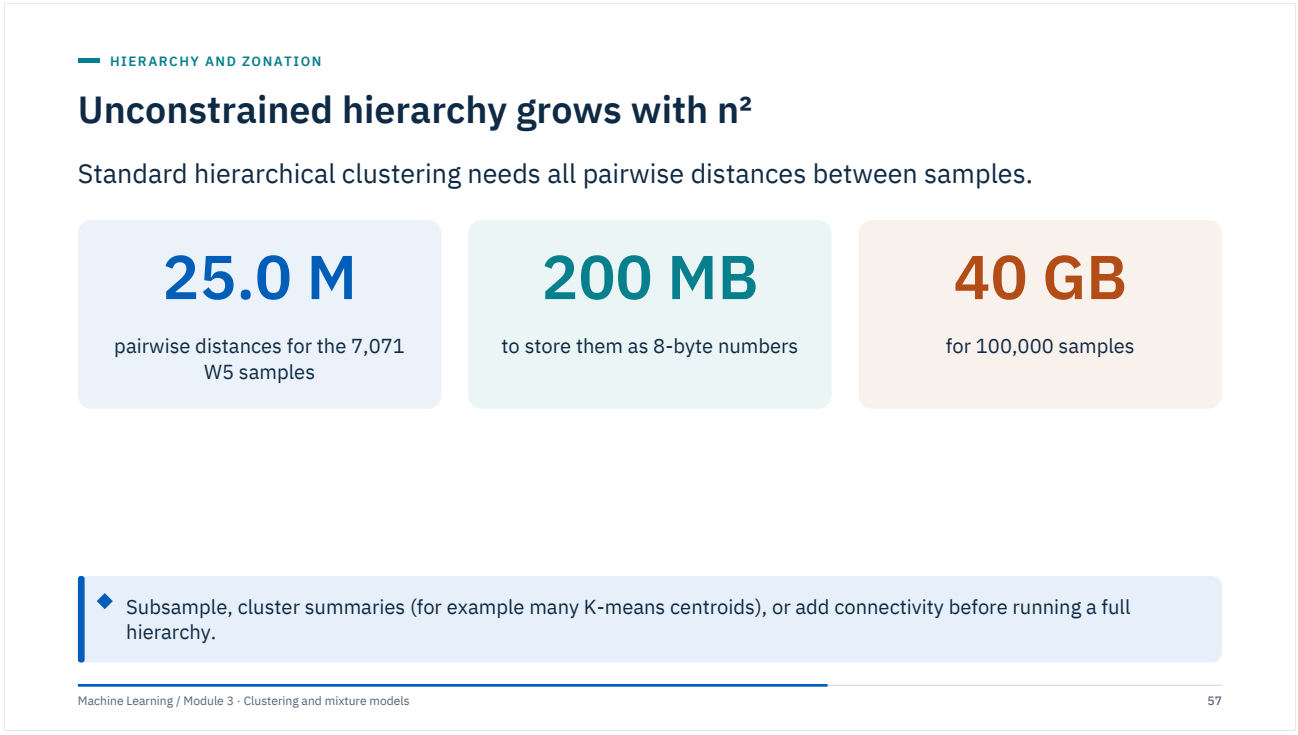
Line by line. Sort the samples by depth so that neighbouring rows are neighbouring depths. Standardize the logs (log10 of resistivity was computed beforehand as LOG_RT). The sparse matrix A has ones just above and below the diagonal: sample i is connected to samples i − 1 and i + 1 only. Passing it as connectivity restricts every merge to adjacent clusters, so every cluster is a contiguous depth interval.

For several wells, build a block-diagonal matrix so that no zone crosses from one well to another, and set the entries across unlogged gaps to zero.

Change n_clusters to get a coarser or finer zonation, or set n_clusters=None and a distance_threshold to cut at a height instead.

With a connectivity matrix scikit-learn only considers merges between connected clusters, which is also much faster than the unconstrained algorithm on long logs. [2 min]

Unconstrained hierarchy grows with n²



The number of pairs is $n(n-1)/2$. For the 7,071 W5 samples that is 24,995,985 distances, about 200 MB as 8-byte floating-point numbers. For 100,000 samples, a modest amount of high-frequency sensor data, it is about 5.0×10^9 distances or 40 GB, beyond a laptop. Run time also grows at least with n^2.

Practical options: subsample and then assign the remaining samples to the nearest cluster; summarize first, for example K-means with a few hundred centroids followed by a hierarchy on the centroids; use a connectivity matrix, as in zonation, which limits merges to neighbours; or use BIRCH, which builds a compact summary tree in one pass (mentioned again in Part 6).

Ask: roughly how many samples would a 16 GB laptop handle for an unconstrained hierarchy, ignoring overheads? Answer: $8n^2/2 \approx 16 \times 10^9$ gives $n \approx 63{,}000$. [2 min]

Which linkage for which task?

PRACTICE

4 min Pairs

Which linkage for which task?

Choose a linkage or constraint for each task and justify it.

1. Separate two dense sand bodies joined by a thin streak of transitional samples.
2. Follow a long, narrow channel feature traced in a seismic attribute map.
3. Build contiguous layers from logs for a reservoir model.

Give three minutes. [4 min]

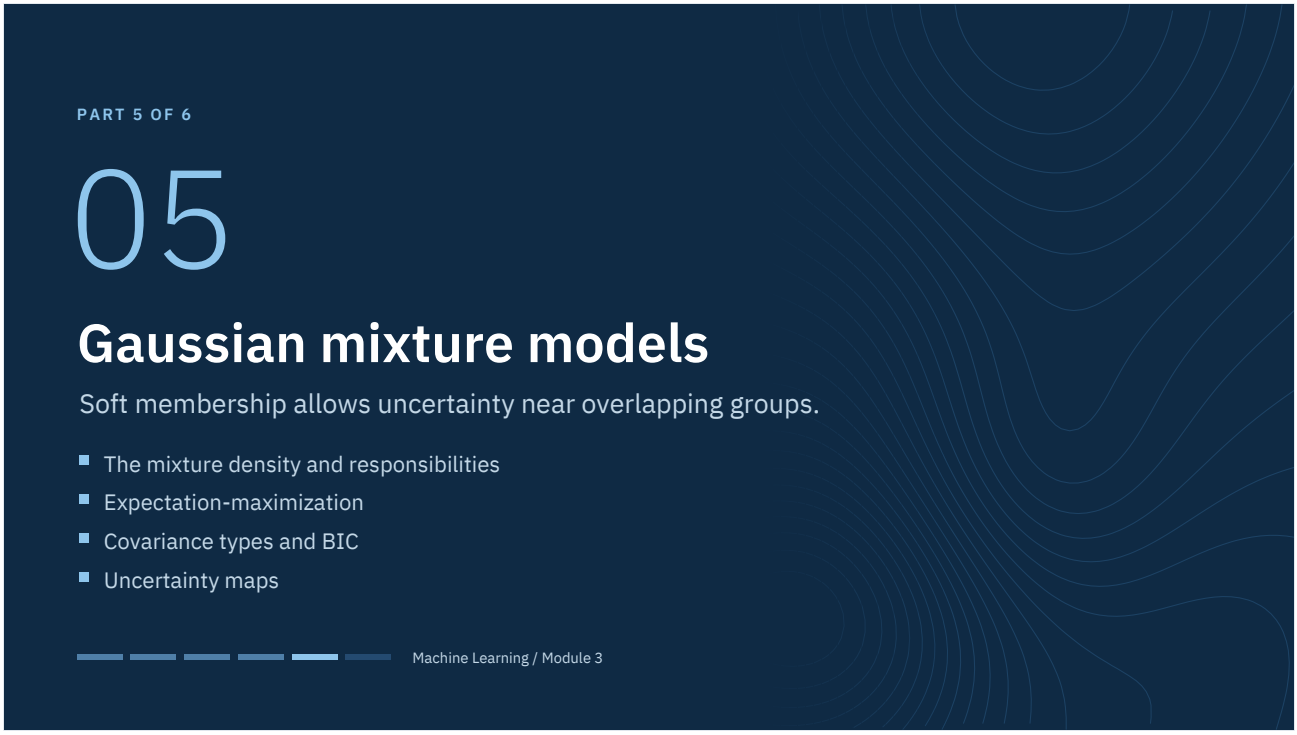
Worked answer. Task 1: avoid single linkage, which would chain through the transitional streak. Ward, average or complete linkage keep compact groups apart; Ward is the natural default on standardized Euclidean features.

Task 2: here chaining is desirable, because the feature is long and connected rather than compact. Single linkage, or a density method such as DBSCAN, follows connected points; Ward would chop the channel into compact pieces.

Task 3: Ward with a depth connectivity matrix, so that every cluster is a contiguous interval, cut at the number of layers the model needs or at a large gap in merge heights. Check the boundaries against the logs and against neighbouring wells.

Common error: assuming one linkage is best in general. The right linkage follows the shape you expect, just as the right distance follows the decision.

Gaussian mixture models



Explain a mixture as a weighted sum of Gaussian densities: the data are modelled as if each observation came from one of K Gaussian components, chosen at random with probabilities given by the mixing weights. A mixture component is a statistical distribution, not necessarily a separate physical population.

Mixtures extend K-means in two directions: components can be elongated and tilted (covariance), and membership is soft (responsibilities). The price is more parameters, which is why model selection with BIC takes a central place in this part.

Lab 3 applies all of this to rock geochemistry, so keep a list of the settings that matter: number of components, covariance type, n_init and reg_covar. [1 min]

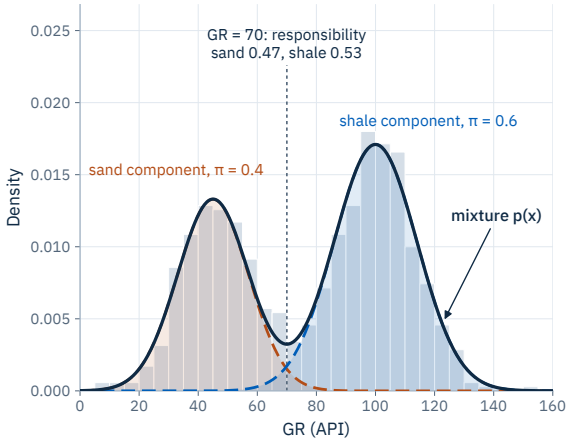
The mixture density adds weighted Gaussians

GAUSSIAN MIXTURE MODELS

The mixture density adds weighted Gaussians

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

- π_k: mixing weights, non-negative, summing to one
- μ_k and Σ_k: mean and covariance of component k
- **Responsibility** r_{ik}: component k's share of the density at x_i



Exact two-component normal mixture (weights 0.4 and 0.6; means 45 and 100 API; SD 12 and 14 API) with a synthetic histogram (seed 9).

Machine Learning / Module 3 · Clustering and mixture models 60

Read the figure. The dashed curves are the two weighted components, sand and shale; the solid curve is their sum, the mixture density p(x). The histogram shows 700 simulated samples from this mixture.

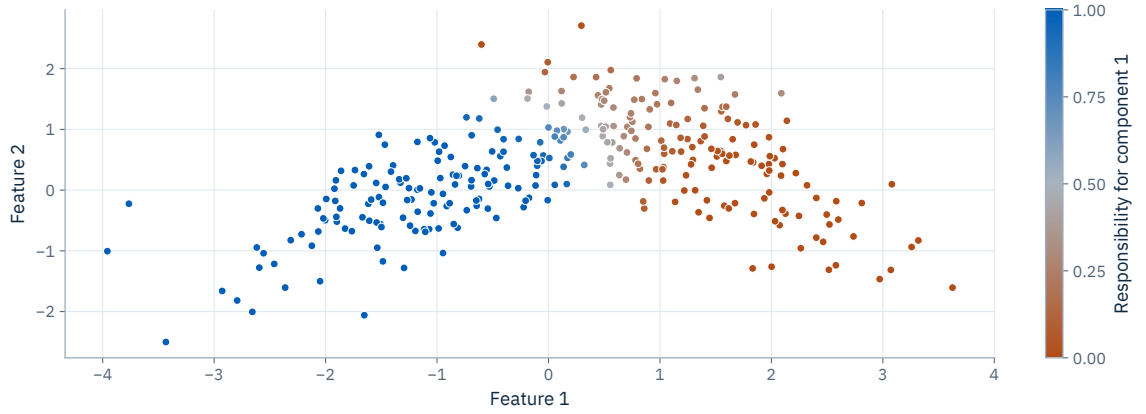
The responsibility of component k for observation i is r_ik = π_k N(x_i | μ_k, Σ_k) divided by the sum of the same term over all components j: the weighted density of component k as a share of the total density at x_i. At GR = 70 API the two weighted densities are almost equal, so the responsibilities are 0.47 for sand and 0.53 for shale; at 60 API sand takes 0.96, and at 80 API only 0.03.

Distinguish a density value from a probability: p(x) has units of 1/API and can exceed 1 for narrow components. Responsibilities are normalized posterior component probabilities under the fitted model. They are not automatically calibrated probabilities of geological facies: if the model is wrong, the responsibilities are confidently wrong. [3 min]

Soft assignments expose overlap

GAUSSIAN MIXTURE MODELS

Soft assignments expose overlap



Synthetic overlapping groups (seed 42); two-component Gaussian mixture. Colour: responsibility for component 1; grey points are near 0.5, the ambiguous ones.

Machine Learning / Module 3 · Clustering and mixture models 61

Point to observations near the overlap and compare their uncertainty with distant observations. Here about a quarter of the points (27%) have a responsibility between 0.1 and 0.9 for component 1: they are genuinely ambiguous under the model.

Component labels may swap between fits without changing the model, so never compare raw component numbers across runs. A maximum-responsibility hard assignment (predict) discards the uncertainty; keep predict_proba when the downstream use can handle it.

Relation to K-means: K-means behaves like a mixture with equal, spherical covariances in the limit where the variance shrinks to zero, so that every responsibility becomes 0 or 1. The mixture adds shape and softness. [2 min]

Expectation-maximization alternates two steps

GAUSSIAN MIXTURE MODELS

Expectation-maximization alternates two steps

Step	Quantity updated	Intuition
E step	Responsibilities	How much does each component explain each observation?
M step	Weights, means, covariances	Refit components using fractional memberships

Repeat until the likelihood improvement is small, with safeguards such as reg_covar.

EM (Dempster, Laird and Rubin, 1977) alternates like K-means. The E step computes responsibilities from the current parameters, as on the previous slides. The M step re-estimates each component as a weighted average: weights are the effective counts divided by n, means are responsibility-weighted means, and covariances are responsibility-weighted covariances.

With exact steps the log-likelihood increases or stays the same at every iteration, but EM can settle at a local optimum, so several initializations (n_init) matter, just as for K-means. In scikit-learn the default initialization runs K-means first.

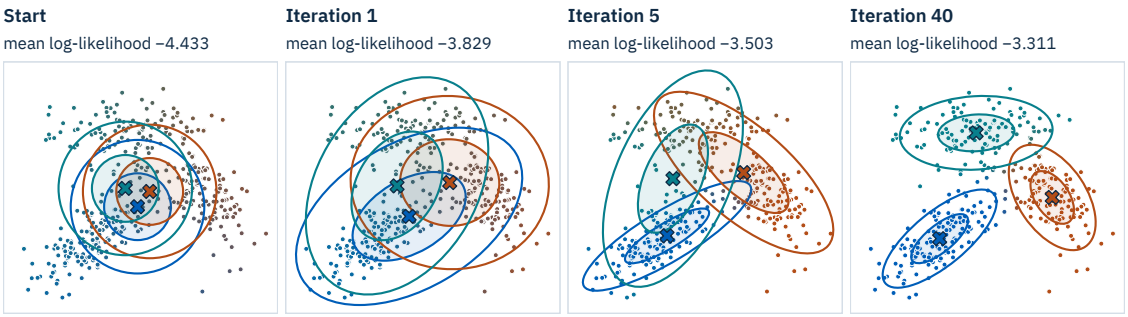
A known failure: a component can collapse onto a few nearly identical points, driving its variance to zero and the likelihood to infinity. reg_covar adds a small constant to the covariance diagonal to prevent this; sensible model complexity helps too. [2 min]

EM moves the components until the likelihood levels off

GAUSSIAN MIXTURE MODELS

EM moves the components until the likelihood levels off

E step: recompute responsibilities. M step: refit weights, means and covariances.



Synthetic three-component data (seed 31); EM for a full-covariance mixture written in numpy from a deliberately poor start. Point colour blends component colours by responsibility.

The start (left) puts three round components on top of each other near the centre; every point has mixed colour because all responsibilities are similar.

After one iteration the components have already stretched towards different parts of the data. By iteration 5 each has found its group, though the teal and orange ellipses are still too wide. By iteration 40 the components match the three generating groups, and point colours are pure except near the boundaries.

The mean log-likelihood per observation rises at every iteration: −4.433 at the start, −3.829 after one iteration, −3.503 after five and −3.311 after forty. The gain per iteration falls below 0.001 after 29 iterations; scikit-learn stops at that kind of point, because its default tol = 0.001 applies to the per-sample lower bound.

Ask: from this start EM found the right answer; what would make it fail? Answer: an unlucky start where two components share one group, the same local-minimum problem as K-means. [3 min]

A weighted mean update

PRACTICE

4 min · Individual

A weighted mean update

$$\mu_k^{\text{new}} = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}$$

Two observations, x = 0 and x = 10, have responsibilities 0.8 and 0.2 for component 1.

1. Compute the new mean of component 1.
2. Compute the new mean of component 2, whose responsibilities are 0.2 and 0.8.
3. What are the new mixing weights, and the new variance of component 1?

Machine Learning / Module 3 · Clustering and mixture models 64

Give three minutes. [4 min]

Worked answer. Component 1: (0.8 × 0 + 0.2 × 10) / (0.8 + 0.2) = 2 / 1 = 2. Component 2: (0.2 × 0 + 0.8 × 10) / (0.2 + 0.8) = 8.

Mixing weights: the effective count of each component is the sum of its responsibilities, 1.0 for each, so each weight is 1.0 / n = 1.0 / 2 = 0.5.

Variance of component 1, with the same weights: (0.8 × (0 − 2)² + 0.2 × (10 − 2)²) / 1.0 = (3.2 + 12.8) / 1.0 = 16, a standard deviation of 4. Component 2 has the same variance by symmetry.

Point out that a hard assignment would put the component 1 mean at 0 and its variance at 0: the fractional membership of the far point keeps the component wide. That is both the strength (honest uncertainty) and the risk (one extreme point with small responsibility can still pull a component).

Common error: dividing by n instead of by the effective count.

Covariance structure controls flexibility

GAUSSIAN MIXTURE MODELS

Covariance structure controls flexibility

Type	Assumption	Covariance parameters	Trade-off
Spherical	One variance per component	K	Few parameters, round groups
Diagonal	No within-component correlation	K·d	Axis-aligned ellipses
Tied	One shared full covariance	d(d+1)/2	Same shape and tilt for all
Full	Own full covariance each	K·d(d+1)/2	Flexible, needs more data

Add K·d mean parameters and K – 1 weights to get the total that BIC penalizes.

Here d is the number of features and K the number of components.

Spherical: each component has one variance, so it is a round blob, much like K-means but with soft membership and different sizes. Diagonal: one variance per feature and component, giving ellipses aligned with the axes; correlations between logs are ignored within a component. Tied: all components share one full covariance, so they have the same shape and orientation. Full: each component has its own d(d+1)/2 covariance parameters.

Full covariance grows quickly with d: for 10 features it is 55 parameters per component, versus 10 for diagonal. With few samples and many features the estimates become unstable, and regularization (reg_covar) or a simpler type is safer.

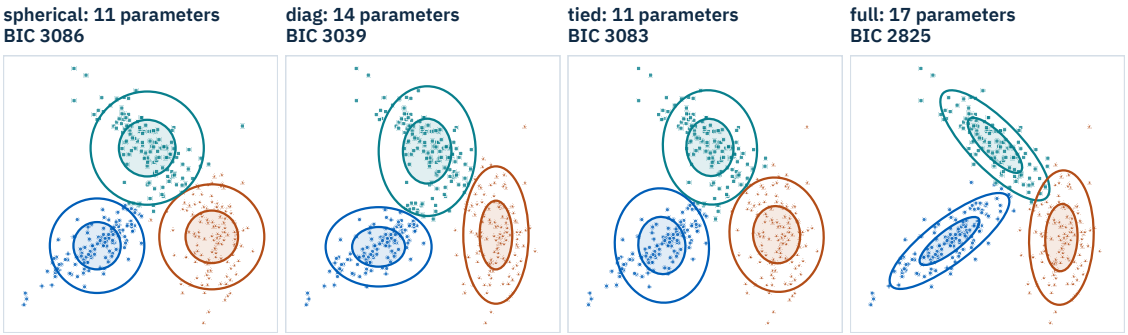
Scale and dimension affect covariance estimation, so standardize or transform features before fitting, as for every other method today. [2 min]

Covariance type sets the shape of each component

GAUSSIAN MIXTURE MODELS

Covariance type sets the shape of each component

Full covariance costs 3 more parameters than diagonal, yet BIC prefers it clearly.



Synthetic three-component data (seed 17); GaussianMixture(K = 3, n_init = 5, random_state = 0) with each covariance_type; ellipses at 1 and 2 standard deviations.

The same data, four covariance types. The generating groups are a tilted ellipse (lower left), a narrow vertical ellipse (right) and an ellipse tilted the other way (top).

Spherical and tied models force round or identical shapes, so they misfit the tilted groups. Diagonal can stretch along the axes and captures the vertical group, but not the tilts. Only full covariance matches all three shapes.

BIC agrees: 3,086 for spherical, 3,039 for diagonal, 3,083 for tied and 2,825 for full (lower is better). The full model has 17 free parameters against 11 to 14 for the others, but its better fit more than pays for the penalty. Its adjusted Rand index against the generating groups is 0.92, against 0.73 to 0.80 for the others.

On real data the best type is not known in advance; compare them with BIC and stability, as Lab 3 asks. [2 min]

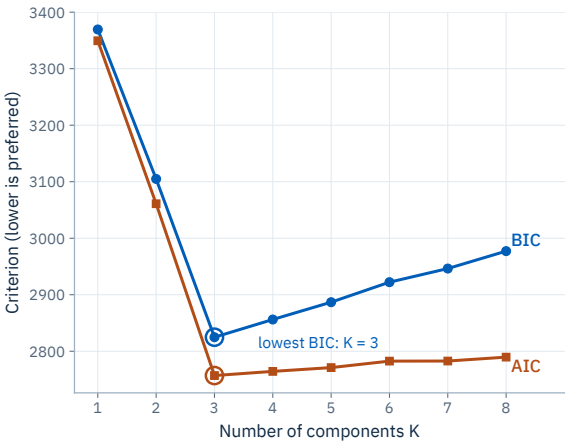
BIC and AIC penalize model complexity

GAUSSIAN MIXTURE MODELS

BIC and AIC penalize model complexity

$$\mathrm{BIC} = -2 \log \hat{L} + p \log n$$

- $\hat{L}$: maximized likelihood; p: free parameters; n: observations
- Lower is preferred; compare models fitted to the same data and features
- AIC replaces $p \log n$ with $2p$, a lighter penalty
- Check stability and plausibility alongside the criterion



Same synthetic data as the covariance figure (three generating components); full-covariance mixtures, K = 1 to 8.

Machine Learning / Module 3 · Clustering and mixture models 67

L̂ (L-hat) is the likelihood at the fitted parameters, n is the number of observations (400 here) and p the number of free parameters; log is the natural logarithm. For this two-dimensional full-covariance mixture, p = 6K − 1: two means, three covariance terms and one weight per component, minus one because weights sum to one.

Both criteria reach their minimum at K = 3, the number of generating components. After that BIC rises steadily because every extra component adds 6 parameters (two means, three covariance terms, one weight) at a penalty of 6 × ln(400) ≈ 35.9, while AIC stays nearly flat because its penalty is only 2 × 6 = 12 per component. In general AIC tends to choose more components than BIC.

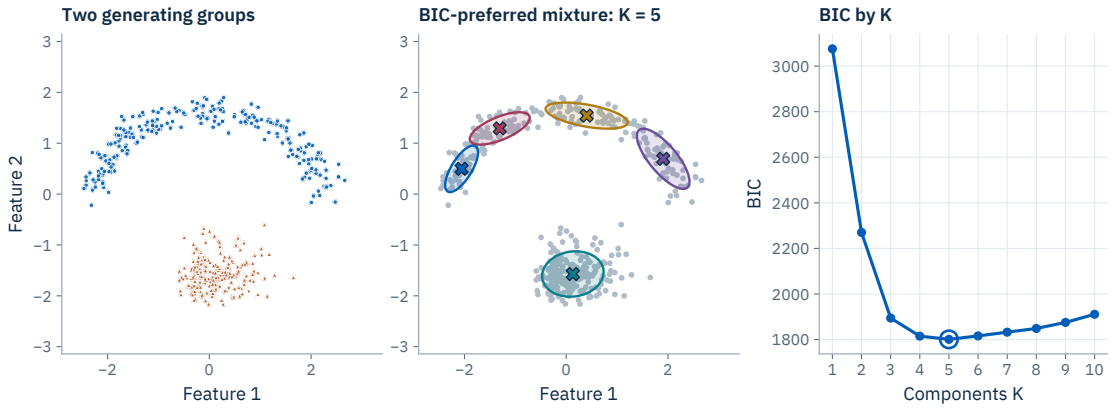
A lower information criterion does not prove that K corresponds to the number of physical facies, as the next slide shows. Avoid comparing BIC values across different preprocessing transformations: the likelihood is measured in the coordinate system of the transformed features, so BIC values from standardized and log-transformed data are not comparable. [3 min]

Mixture components are not the same as groups

GAUSSIAN MIXTURE MODELS

Mixture components are not the same as groups

Two generating groups, but BIC prefers five components: Gaussians tile the curved group.



Synthetic: a curved group and a log-normal (skewed) group, seed 23; full-covariance mixtures, n_init = 5; ellipses at 1.5 standard deviations.

Machine Learning / Module 3 · Clustering and mixture models 68

The data have two groups: a curved arc and a skewed blob. A single Gaussian cannot describe the arc, so the best-fitting mixture places four elongated components along it and one on the blob. BIC is lowest at K = 5 (1,801), with K = 4 (1,815) and K = 6 (1,816) close behind.

BIC answered its own question correctly: how many Gaussians describe this density well. It did not answer ours: how many groups are there.

The same happens in logs. A single rock type whose response changes gradually with compaction or shale content will be described by several components. Before naming components, look at them in original units and along depth; consider merging adjacent components that together form one continuous group, or use a density method for curved shapes.

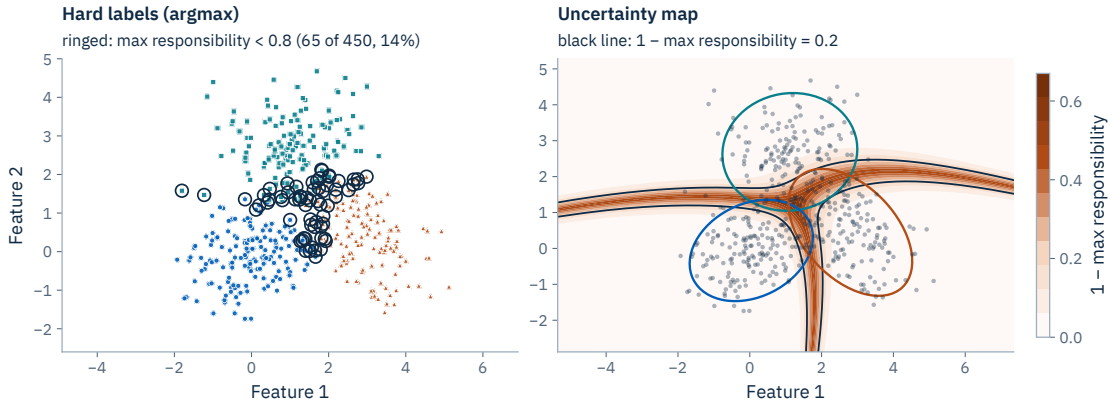
Question: what would a mixture do with a single log-normal permeability distribution? Answer: fit several components to describe its skew, unless you take the logarithm first. [2 min]

Uncertainty maps show where labels are fragile

GAUSSIAN MIXTURE MODELS

Uncertainty maps show where labels are fragile

Responsibilities stay low along boundaries, where a hard label hides ambiguity.



Synthetic overlapping groups (seed 41); GaussianMixture(K = 3, full, n_init = 5, random_state = 0); 2-SD ellipses of the fitted components.

Machine Learning / Module 3 · Clustering and mixture models 69

Left: the hard labels from predict. Ringed points have a maximum responsibility below 0.8: 65 of 450, or 14%. They all lie between groups.

Right: the uncertainty 1 − max responsibility over the whole plane. It is near zero inside each component and rises to its maximum along the boundaries; the black contour marks 0.2. The bands extend beyond the data, where the model extrapolates, which is another reason to treat responsibilities far from the training data with care.

In a well, plot 1 − max responsibility as a track against depth. High values typically flag bed boundaries, thin beds below tool resolution and transitional lithologies, exactly where a petrophysicist would want to look more closely.

Practical rule: carry the probabilities into downstream work (for example as soft facies proportions) rather than discarding them after argmax. [2 min]

A reproducible mixture fit

GAUSSIAN MIXTURE MODELS

A reproducible mixture fit

```python
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X_train)
Z_train = scaler.transform(X_train)
gmm = GaussianMixture(
    n_components=3, covariance_type="full",
    n_init=5, reg_covar=1e-6, random_state=42
).fit(Z_train)
membership = gmm.predict_proba(scaler.transform(X_new))
```

Machine Learning / Module 3 · Clustering and mixture models 70

Explain that the example assumes an explicit future-data use case: the scaler and the mixture are fitted on X_train and then applied to X_new. For a purely exploratory description of one fixed dataset, fitting on all available observations is valid, but it cannot support a held-out generalization claim. X_new must use the same feature order and units.

The settings: n_components is K; covariance_type one of "full", "tied", "diag" or "spherical"; n_init = 5 runs EM from five initializations and keeps the best likelihood; reg_covar = 1e-6 (the default) is added to the covariance diagonals for numerical stability; random_state makes the result repeatable.

predict_proba returns an n × K array of responsibilities whose rows sum to one. Check gmm.converged_ after fitting, and compare gmm.bic(Z_train) across candidate models fitted to the same Z_train. [2 min]

Count the parameters before you fit

PRACTICE

5 min Pairs

Count the parameters before you fit

Lab 3 fits mixtures to d = 10 standardized oxides on n = 1,684 samples.

1. How many free parameters does a full-covariance mixture with K = 8 have?
2. How many does a diagonal-covariance mixture with K = 8 have?
3. By how much must the full model raise the log-likelihood to beat diagonal on BIC?

Machine Learning / Module 3 · Clustering and mixture models 71

Give four minutes; calculators allowed. [5 min]

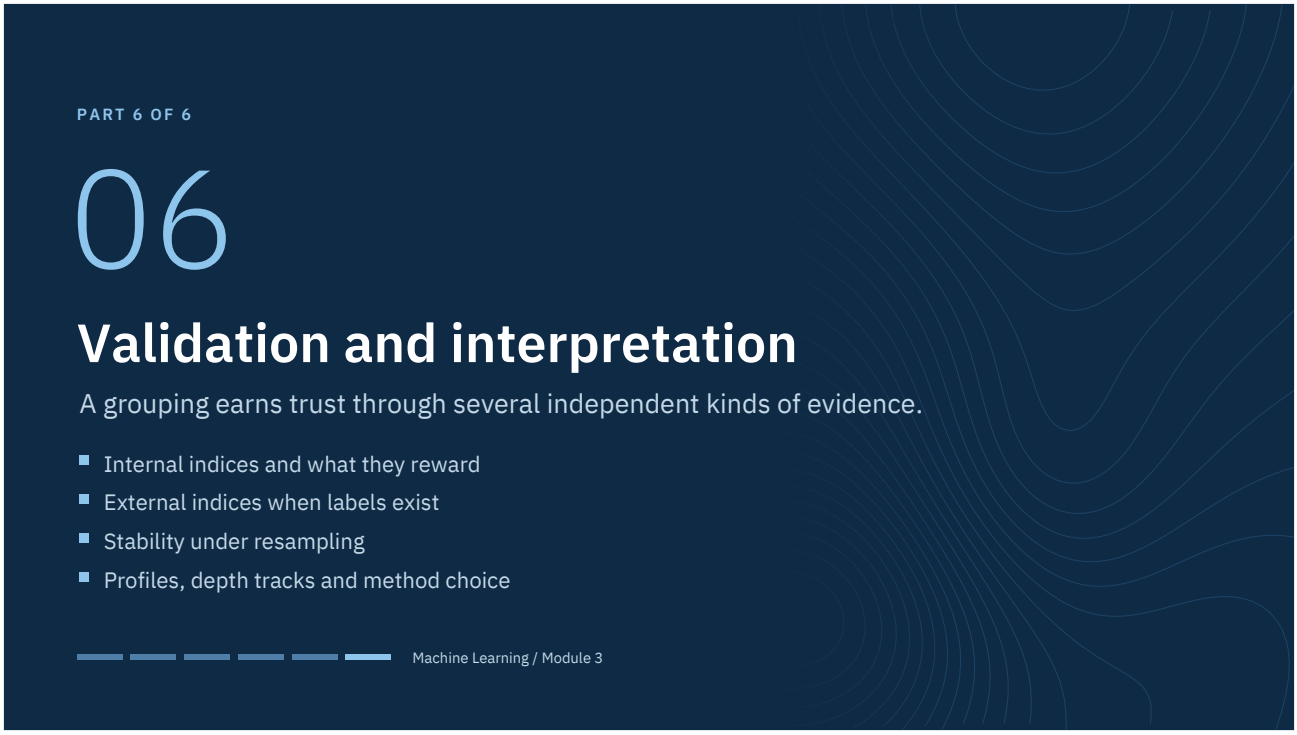
Worked answer. Full: means K·d = 80; covariances K·d(d+1)/2 = 8 × 55 = 440; weights K − 1 = 7; total 527. Diagonal: means 80, variances K·d = 80, weights 7; total 167. scikit-learn reports the same counts.

BIC penalty difference: (527 − 167) × ln(1,684) = 360 × 7.429 = 2,674. BIC = −2 log L + p ln n, so the full model is preferred only if −2 log L falls by more than 2,674, that is, if the log-likelihood rises by more than 1,337.

Discussion: that is a large requirement, but with 1,684 samples and strongly correlated oxides it can easily be met. The question to keep in mind for the lab is different: even if BIC prefers the full model, does it produce components that correspond to rock types?

Common errors: forgetting that weights sum to one (K − 1, not K); counting d^2 instead of d(d+1)/2 covariance terms.

Validation and interpretation



We now step back from algorithms to evidence. Every method today returns a grouping; the question in this part is how to decide whether a grouping is stable, meaningful and useful.

Four kinds of evidence, from weakest to strongest for a physical interpretation: internal indices computed from the data alone; stability under perturbation; agreement with independent labels such as core facies; and a coherent interpretation in original units and in space or time that a domain expert can test.

None of them is sufficient alone, which is the message of the "Scores need context" table and of the final exercise of this part. [1 min]

Internal indices reward different geometry

VALIDATION AND INTERPRETATION

Internal indices reward different geometry

Index	Rewards	Better	Caveat
Silhouette	Points nearer their own cluster than the next	Higher, −1 to 1	Needs all pairwise distances
Davies–Bouldin	Small spread relative to centroid separation	Lower, 0 is best	Centroid-based: favours convex
Calinski–Harabasz	High between- to within-cluster variance	Higher	Favours convex, similar-size

All three use only the data and the labels, so they share the clustering's own assumptions.

Silhouette (Rousseeuw, 1987) averages the per-sample values from Part 2. Davies–Bouldin (1979) averages, over clusters, the worst ratio of within-cluster spread to the distance between centroids; lower is better and 0 is the minimum. Calinski–Harabasz (1974) is the ratio of between-cluster to within-cluster dispersion, scaled for the number of clusters and samples; higher is better, and there is no upper bound.

All three are available in sklearn.metrics as silhouette_score, davies_bouldin_score and calinski_harabasz_score.

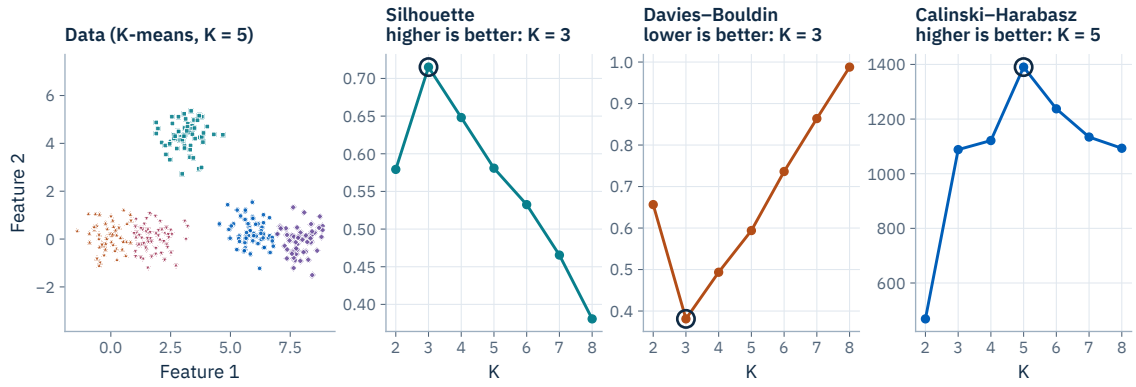
The key caveat: they reward compact, well-separated, convex clusters, so they will favour K-means-like solutions and penalize a correct DBSCAN result on curved shapes. They cannot tell you whether the clusters are physically meaningful, only whether they are geometrically tidy in the chosen feature space. [2 min]

Internal indices can disagree on K

VALIDATION AND INTERPRETATION

Internal indices can disagree on K

Silhouette and Davies–Bouldin see three groups; Calinski–Harabasz resolves all five blobs.



Synthetic five blobs with two close pairs (random_state = 0); K-means (n_init = 10) for K = 2 to 8; scikit-learn metric implementations.

Machine Learning / Module 3 · Clustering and mixture models 74

The data are five blobs, two of which form close pairs. Silhouette is highest at K = 3 (0.72) and Davies–Bouldin lowest at K = 3 (0.38): both prefer treating each close pair as one group. Calinski–Harabasz is highest at K = 5 (1,390, against 1,088 at K = 3): it prefers resolving every blob.

Which is right? Both: the data have structure at two scales. A coarse description has three groups and a fine description five. The indices differ in how they trade cohesion against separation, not in correctness.

This is common in subsurface data, where rock types nest inside broader groups, which is why hierarchical methods and nested zonations are useful. Report the index values for several K, explain the scale at which you are working, and let stability and domain evidence decide. [2 min]

Scores need context

VALIDATION AND INTERPRETATION

Scores need context

Evidence	What it can establish	What it cannot establish alone
Silhouette	Separation under a selected distance	Geological validity
Adjusted Rand index	Agreement with an external labelling	Correctness of that labelling
Stability across fits	Robustness to perturbations	Usefulness for a decision
Expert review	Domain plausibility	Generalization to all new data

Machine Learning / Module 3 · Clustering and mixture models 75

Walk down the table. Each kind of evidence answers a narrow question and leaves a gap that another kind must fill.

Align cluster labels before comparing raw assignments, or use label-invariant metrics such as the adjusted Rand index. Use a held-out labelled set only for external evaluation when the labels were not already used to choose the clustering; if you tuned K or features to maximize agreement with core facies, that agreement is no longer independent evidence.

Stability shows that a result is not a fluke of sampling or initialization, but a stable artefact (for example a split by well caused by a calibration difference) is still an artefact. Expert review adds plausibility but may be anchored on expectations.

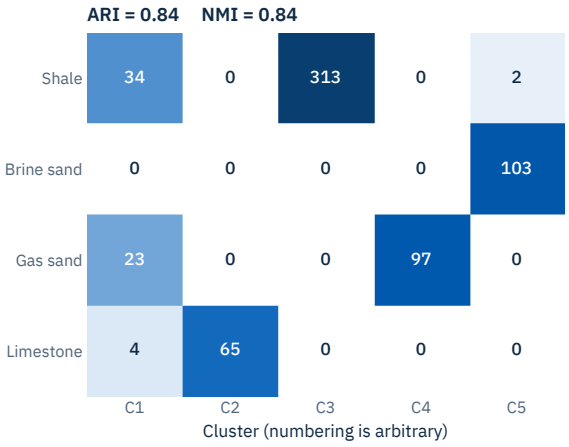
Transition: the next slides show how to compute the middle two rows. [2 min]

External indices compare clusters with reference labels

VALIDATION AND INTERPRETATION

External indices compare clusters with reference labels

- **ARI**: pair agreement corrected for chance; 0 random, 1 identical
- NMI: shared information, 0 to 1; not chance-corrected (AMI is)
- Both ignore cluster numbering; neither says which cluster is wrong
- Use labels to evaluate, never as clustering features



Synthetic well S-1 (seed 7): reference lithology against K-means (K = 5) on standardized logs; counts of depth samples.

Machine Learning / Module 3 · Clustering and mixture models 76

The adjusted Rand index (Hubert and Arabie, 1985) counts pairs of samples that the two labellings treat the same way (together in both, or apart in both), corrected so that random labellings score about 0 in expectation; it can be slightly negative. Normalized mutual information measures shared information on a 0-to-1 scale but is not corrected for chance; adjusted_mutual_info_score is the corrected version.

On well S-1 with K = 5 both give 0.84. The contingency table explains why it is not higher: four clusters match the four lithologies, and the fifth, C1, collects 61 samples of mixed response: 32 from the thin-bedded unit, 28 within a metre of a bed boundary and one in the gradational unit. That is a useful discovery (transitional or unresolved samples), not an error, but the single index cannot say so.

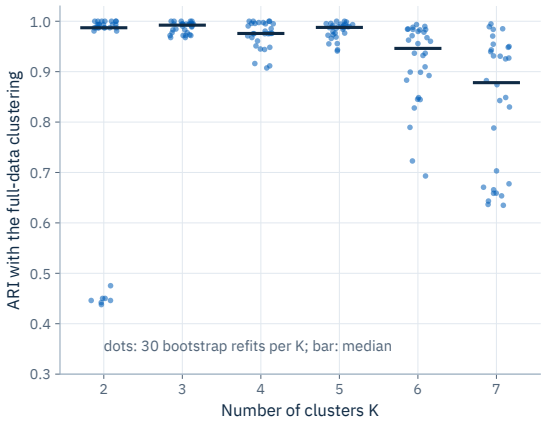
Always show the contingency table next to ARI or NMI. [3 min]

Stable clusters survive resampling

VALIDATION AND INTERPRETATION

Stable clusters survive resampling

- Refit on bootstrap resamples; compare labels on all samples with ARI
- Keep K, features and scaling fixed
- A split that flips between runs is not a finding
- Resample by well or depth block when samples are correlated



Synthetic well S-1 (seed 7), standardized logs; K-means refitted on 30 bootstrap resamples per K; labels predicted for all samples and compared with the full-data fit.

Machine Learning / Module 3 · Clustering and mixture models 77

For each K: fit on the full data, then refit on 30 bootstrap resamples (sampling rows with replacement), predict labels for all samples with each refit, and compute the ARI with the full-data labels.

K = 3, 4 and 5 are stable: median ARI 0.98 to 0.99, with the worst resample at 0.91 or above. K = 2 is bistable: most resamples agree, but 7 of 30 land near 0.44, meaning two different two-way splits compete. K = 6 and 7 are unstable: medians 0.95 and 0.88, with many resamples below 0.9.

So stability narrows the choice to K = 3 to 5, and domain evidence must choose among them.

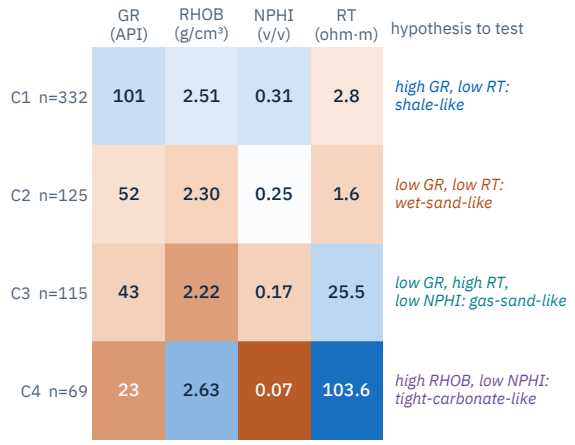
Caution: neighbouring depth samples are strongly correlated, so resampling individual samples overstates stability. For logs, resample blocks of depth or whole wells, which is closer to asking whether the clusters would reappear in a new well. [3 min]

Profile clusters in original units

VALIDATION AND INTERPRETATION

Profile clusters in original units

- Report cluster means or medians in physical units
- Add counts, spreads and depth ranges
- Write each reading as a hypothesis to test
- Check against core, cuttings or tests before naming



Synthetic well S-1: K-means (K = 4). Cell values are cluster means in original units (RT as geometric mean); colour shows the standardized mean.

Machine Learning / Module 3 · Clustering and mixture models 78

Standardized centroids are hard to read, so transform them back or compute means directly in original units, and colour the cells by the standardized value so that highs and lows stand out: blue above the well average, orange below.

Read the table. C1 (332 samples): GR 101 API, RHOB 2.51 g/cm³, NPHI 0.31, RT 2.8 ohm·m, shale-like. C2 (125): GR 52, RHOB 2.30, NPHI 0.25, RT 1.6, a porous, conductive, water-bearing sand-like response. C3 (115): GR 43, RHOB 2.22, NPHI 0.17, RT 25.5; low density with low neutron porosity is the classic gas crossover, so gas-sand-like. C4 (69): GR 23, RHOB 2.63, NPHI 0.07, RT 104, tight-carbonate-like.

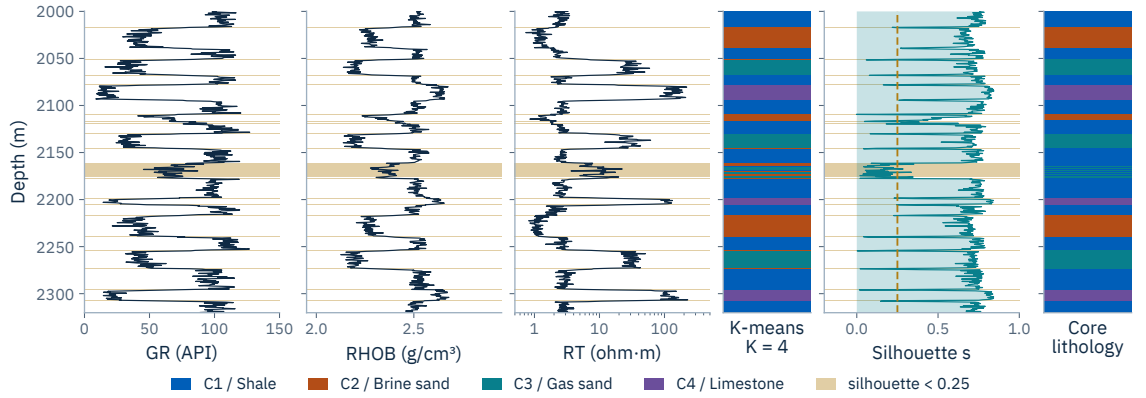
The italic readings are hypotheses. In S-1 they are right because we simulated them; in a real well they need core, cuttings, formation tests or production data. Add spreads (for example interquartile ranges): a mean hides whether a cluster is tight or broad. [3 min]

Map clusters back onto depth

VALIDATION AND INTERPRETATION

Map clusters back onto depth

Low silhouette marks thin beds and boundaries, where one label per sample fails.



Synthetic well S-1 (seed 7): K-means (K = 4) on four standardized logs; per-sample silhouette; core lithology = simulation truth.

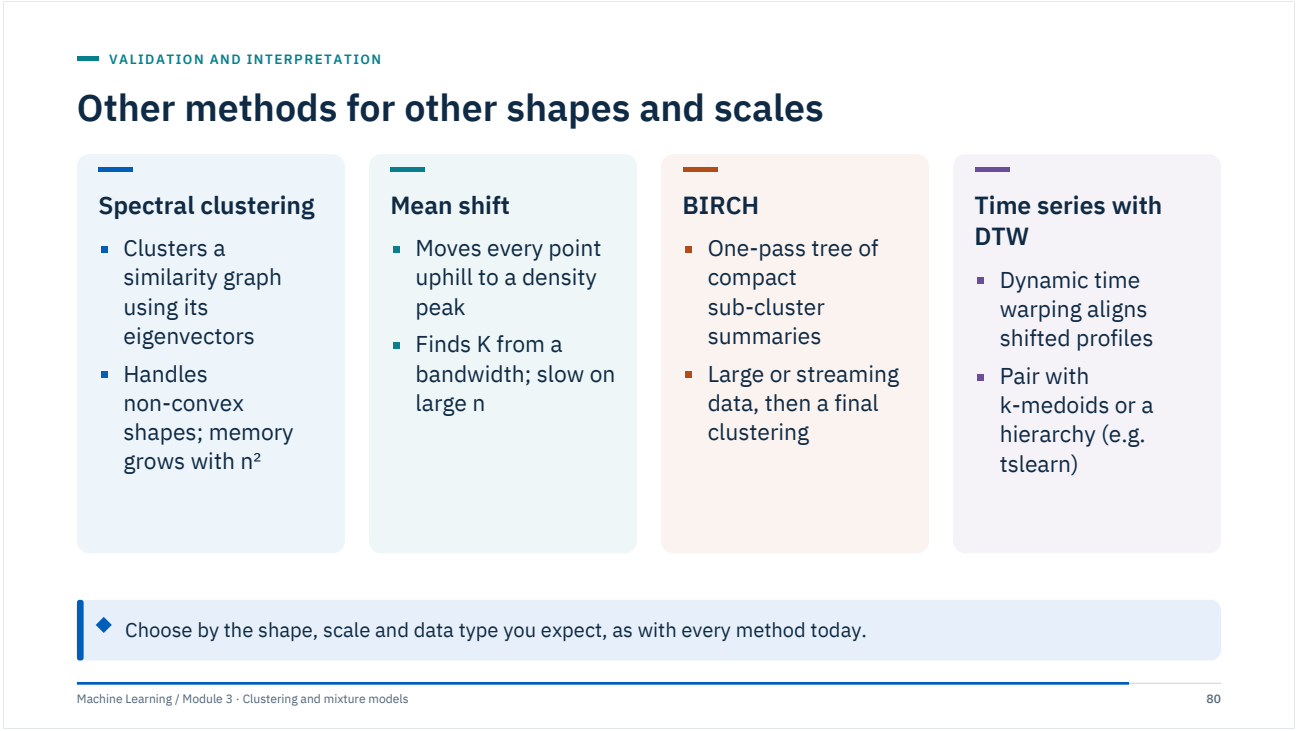
Machine Learning / Module 3 · Clustering and mixture models 79

The log display is where clusters meet geology. From left: gamma ray, density and resistivity; the K-means clusters as a colour track; the per-sample silhouette; and the true lithology of the simulation.

The cluster track reproduces the thick beds almost perfectly. The silhouette track tells us where not to trust it: the mean silhouette is 0.65, but 8% of samples fall below 0.25 (gold bands), and they concentrate at bed boundaries and in the thin-bedded unit between 2,162 and 2,179 m, where 79% of samples are below 0.25. There the logs average several beds within their resolution, and the cluster labels flicker.

In practice, add the depth track to every clustering deliverable, compare it with core and image logs where available, and consider depth constraints (Part 4) or smoothing when the goal is layering rather than facies. [3 min]

Other methods for other shapes and scales



Brief tour, conceptual only.

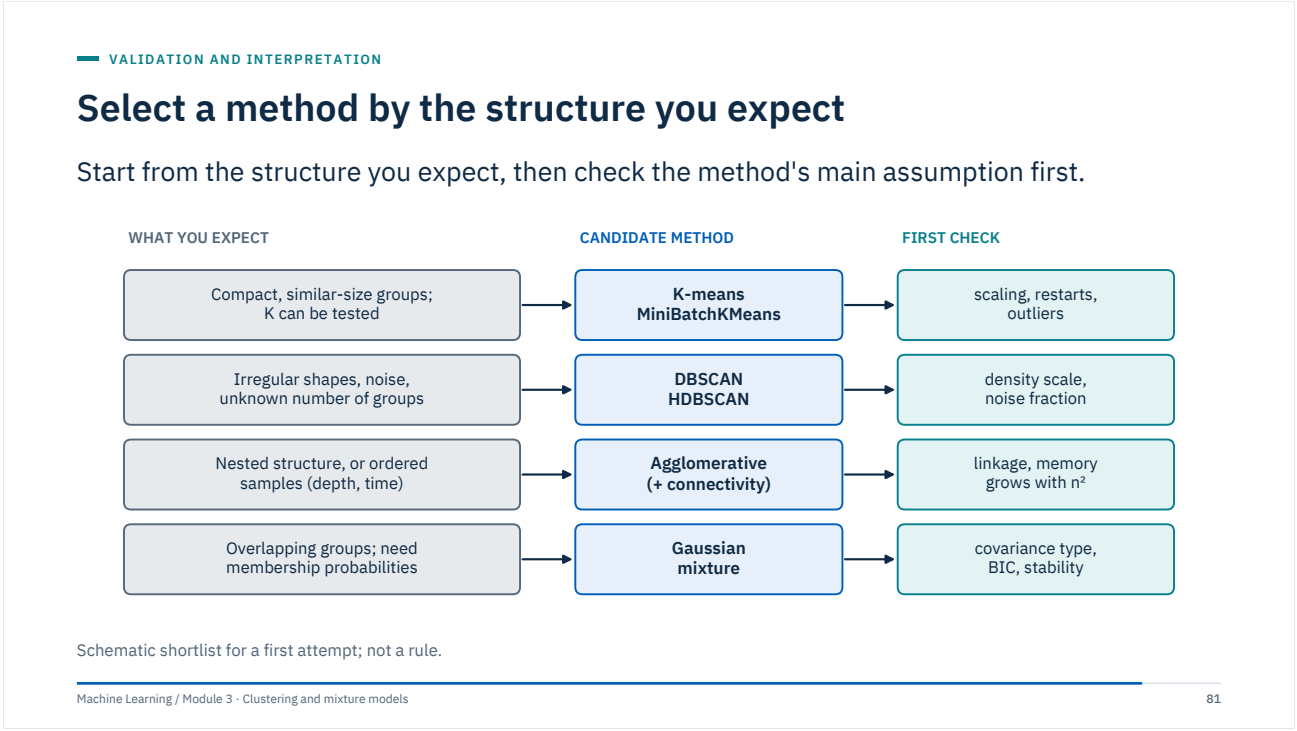
Spectral clustering builds a graph where similar samples are connected, computes eigenvectors of that graph, and clusters in the eigenvector space. It can separate curved and nested shapes, but it needs K and a similarity scale, and storing the graph for large n is costly. SpectralClustering in scikit-learn.

Mean shift moves each point towards the local mean of its neighbours until it reaches a density peak; points reaching the same peak form a cluster. The bandwidth sets the scale. MeanShift in scikit-learn.

BIRCH summarizes the data in one pass into a tree of small sub-clusters, which can then be clustered globally; useful for millions of sensor records. Birch in scikit-learn.

Dynamic time warping aligns two sequences that are shifted or stretched in time, such as production profiles with different ramp-up delays, before measuring distance. It is not in scikit-learn; tslearn offers TimeSeriesKMeans with metric="dtw". Sequence models are developed further in the neural-network course (Day 5, Sequence models and attention). [3 min]

Select a method by the structure you expect



Use this as a hypothesis shortlist. Compact groups where K can be tested: K-means, with scaling, restarts and outliers as the first checks. Irregular shapes with noise and an unknown number of groups: DBSCAN or HDBSCAN, checking the density scale and noise fraction. Nested structure or ordered samples: agglomerative clustering, with connectivity for depth or time, checking linkage and memory. Overlapping groups where membership probabilities are needed: Gaussian mixtures, checking covariance type, BIC and stability.

A method can produce an answer even when its assumptions are poor. Encourage comparing at least two methods with incompatible assumptions on the same scientific question; where they agree, confidence grows, and where they disagree, the disagreement is informative, as with K-means and DBSCAN on W5.

The comparison notebook (exercises/05_comparison) does exactly this and is the extension for Lab 3. [2 min]

Weigh the evidence for a grouping

PRACTICE

6 min Groups

Weigh the evidence for a grouping

K-means on five logs from eight wells gives K = 5, mean silhouette 0.46. Bootstrap by well gives median ARI 0.62. Cluster 4 comes almost entirely from one well. ARI with core facies in two cored wells is 0.35.

1. Which evidence supports the grouping, and which undermines it?
2. What is the most likely explanation for cluster 4?
3. What would you do next?

Give four minutes in groups of three, then discuss. [6 min]

Worked answer. Support: a moderate silhouette (0.46) indicates some separation in feature space. Undermining: stability across wells is low (median ARI 0.62), so the grouping changes when wells are left out; agreement with core facies is weak (0.35); and one cluster is essentially a single well.

Cluster 4 most likely reflects a well-specific difference rather than a rock type: a tool or calibration difference, a missing environmental correction (for example borehole size or mud weight), different mud or invasion, or a genuinely different stratigraphic position. Check that well's log distributions against its neighbours.

Next steps: normalize logs between wells (for example a two-point gamma-ray normalization on shale and clean sand levels), apply environmental corrections, drop features that carry well identity, rerun with stability assessed by leaving out whole wells, and only then compare with core again. Report the result as a candidate electrofacies scheme, not as facies.

Common error: accepting the silhouette as sufficient because it is "good enough".

Lab 3: Gaussian mixtures on rock geochemistry

LABORATORY

40 min

Lab 3: Gaussian mixtures on rock geochemistry

- ☐ Read the missing-value report (1,684 of 2,191 rows remain), then standardize.
- ☐ Fit a 3-component mixture; inspect predict and predict_proba.
- ☐ Compare BIC and AIC for K = 1–10 and refit at your chosen K.
- ☐ Crosstab components against ROCK1, compute ARI, and compare covariance types.
- ☐ Extension: the hierarchical (04) or method-comparison (05) notebook.

NOTEBOOK

`exercises/03_gmm/GMM_Rock_Decomposition_Student.ipynb`

EVIDENCE TO SUBMIT

- BIC and AIC curves with your chosen K
- A crosstab with ARI and silhouette
- One paragraph on what the components do and do not represent

Machine Learning / Module 3 · Clustering and mixture models 83

Timing: 30 minutes of work and a 10-minute debrief. [40 min]

Missing values: rock_multiclass_raw.csv has gaps in nine of the ten oxide columns (FeOT 326 rows, P2O5 160, MnO 85, fewer elsewhere) and one row without a ROCK1 label, and GaussianMixture does not accept NaN. A provided cell at the start of Section 2, complete in the student notebook too, drops those rows and prints the missing count per column and the class counts before and after. That keeps 1,684 of 2,191 samples and changes the class balance (for example basalt 823 to 670), which is worth noting in the write-up. Imputation is the alternative, but it would invent compositions.

ROCK1 is the reference label: use it only for the crosstab and ARI, never as a feature.

Expected results (standardized, n_init = 5, random_state = 42): BIC is lowest at K = 8; AIC keeps decreasing up to K = 10. The refit at K = 8 (n_init = 10) reaches the same BIC, 18,074.

Common errors: skipping the missing-value cell, so GaussianMixture stops with a NaN error; including ROCK1 in the features; comparing raw component numbers between runs.

Extensions for fast finishers: exercises/04_hierarchical/Hierarchical_Clustering_Student.ipynb applies Ward linkage, dendrogram cuts and a linkage comparison to DTC and RHOB logs, cutting in the largest gap between merge heights (two clusters); exercises/05_comparison/Clustering_Comparison_Student.ipynb runs all four methods on the W5 logs with K = 3 from Lab 1 and eps = 0.5 from the k-distance plot, as in Lab 2.

Lab 3 debrief: BIC fits the density, not the labels

DISCUSSION

Lab 3 debrief: BIC fits the density, not the labels

Covariance type, K = 8	BIC	ARI with rock type	Silhouette
Full	18,074	0.28	0.09
Tied	29,432	0.32	0.25
Diagonal	24,776	0.28	0.17
Spherical	31,000	0.30	0.19

The BIC-preferred model describes the oxide distribution best, yet agrees only weakly with the named rock types.

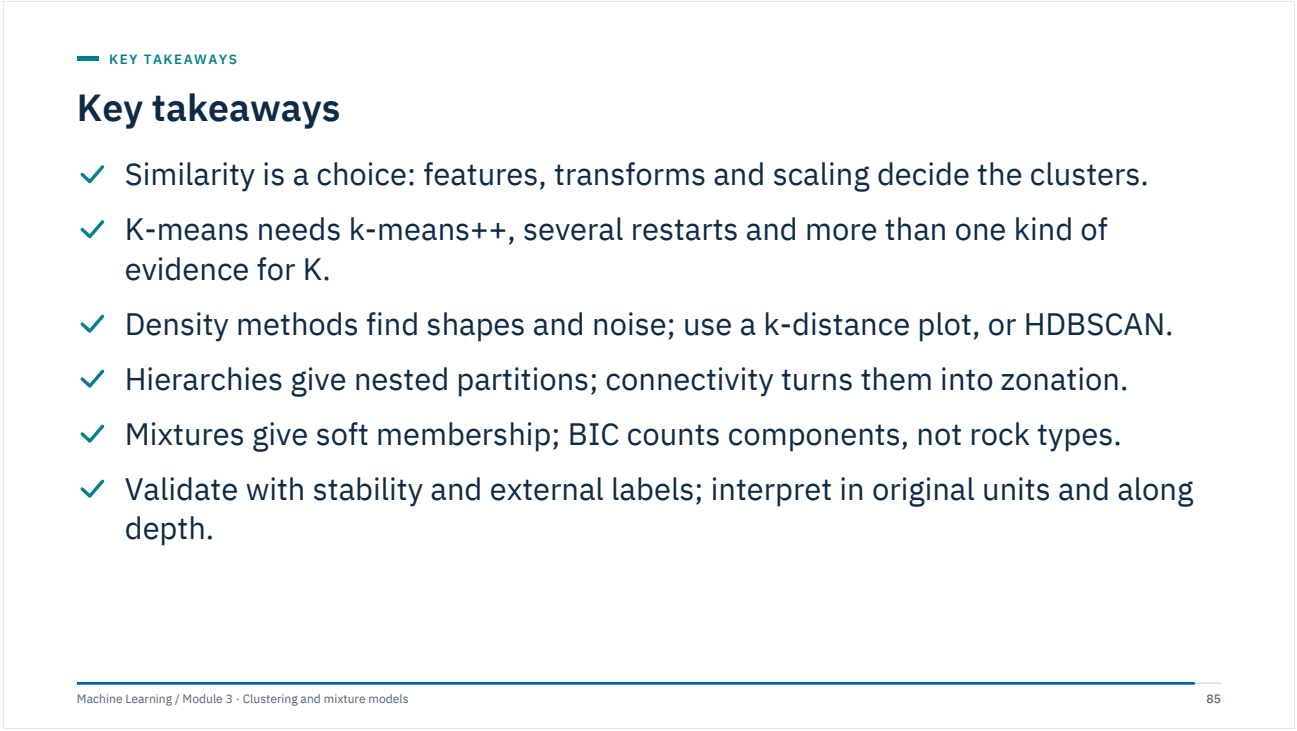
Compare with the groups' results; small differences come from n_init and random_state. [10 min]

BIC by K (full covariance): 36,767 at K = 1 falling to a minimum of 18,074 at K = 8. At K = 8 the full model wins BIC by a wide margin, but its ARI with ROCK1 is only 0.28 and its silhouette 0.09; tied covariance has the highest ARI (0.32) and silhouette (0.25) but by far the worst BIC.

The crosstab explains it. Komatiite (183 of 259 in one component) and rhyolite (229 of 286 in one) are captured well: they are compositional end-members. Basalt spreads over four components (250, 160, 148 and 76 samples), and andesite, tholeiite and gabbro overlap with it. Named rock types are partly defined by criteria other than these oxides: gabbro is the coarse-grained intrusive equivalent of basalt with a similar composition, and tholeiite is a basalt series. The classes form a continuum.

Two further points. Oxide percentages sum to about 100%, so they are compositional data; log-ratio transforms (Aitchison, 1986) are the standard treatment. And a low ARI is not a failure of the method: it tells us the labels and the chemical structure are not the same thing.

Key takeaways



Read each takeaway and connect it to a moment in the day. Scaling: the GR and RHOB example and the W5 depth effect. K-means: the 5-of-20 random starts and the silhouette plots. Density: the k-distance knee and the HDBSCAN comparison. Hierarchy: the zonation of well S-1. Mixtures: five components for two groups, and the rock geochemistry lab. Validation: the stability plot and the depth tracks.

Return to the question asked at the start: how did you decide a clustering result was right? Ask two learners to answer again using today's vocabulary: stability, external labels, profiles in original units, and a named decision.

Remind the room that the next module (dimensionality reduction) builds directly on this one: PCA and manifold methods are often applied before clustering, and they change distances in the same way scaling does. [3 min]

Exit check

PRACTICE

7 min Individual

Exit check

1. Why can scaling change the scientific conclusion of a clustering?
2. Why can DBSCAN noise include valid observations?
3. What information disappears when a mixture becomes a hard partition?
4. Name two independent kinds of evidence you need before calling a cluster a facies.

Machine Learning / Module 3 · Clustering and mixture models 86

Learners write short answers individually, then compare with a neighbour. [7 min]

Expected answers. Scaling changes distances, and distances define every clustering method today, so a change of units, transform or weights changes which samples are similar and therefore the groups; the GR and density example showed a completely different partition.

DBSCAN noise means insufficient density at the chosen eps and min_samples, not invalid data: thin beds, rare lithologies, sparse sampling or a poorly chosen eps all produce noise.

A hard partition discards the fractional membership: the uncertainty near boundaries, the difference between a confident and a marginal assignment, and any information about transitional samples.

Evidence for naming a facies: agreement with core or cuttings descriptions (external labels), stability across wells or resamples, consistent profiles in original units that match rock physics expectations, and a coherent depth pattern. Any two independent ones are acceptable answers; silhouette alone is not.

Ask learners to distinguish a useful descriptive grouping from evidence of distinct physical mechanisms.

References and further study

FURTHER STUDY

References and further study

- scikit-learn User Guide: clustering, Gaussian mixture models, and clustering performance evaluation.
- K-means: Lloyd (1982), least-squares quantization; Arthur and Vassilvitskii (2007), k-means++.
- Density: Ester et al. (1996), DBSCAN; Ankerst et al. (1999), OPTICS; Campello et al. (2013), HDBSCAN.
- Hierarchy and mixtures: Ward (1963); Dempster, Laird and Rubin (1977), EM; Schwarz (1978), BIC.
- Validation: Rousseeuw (1987), silhouettes; Hubert and Arabie (1985), adjusted Rand index; Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, chapter 14.

Machine Learning / Module 3 · Clustering and mixture models 87

Full references. Lloyd, S. P. (1982). Least squares quantization in PCM. IEEE Transactions on Information Theory, 28(2), 129–137. Arthur, D. and Vassilvitskii, S. (2007). k-means++: the advantages of careful seeding. Proceedings of the 18th ACM-SIAM Symposium on Discrete Algorithms, 1027–1035. Ester, M., Kriegel, H.-P., Sander, J. and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. Proceedings of KDD-96, 226–231. Ankerst, M., Breunig, M. M., Kriegel, H.-P. and Sander, J. (1999). OPTICS: ordering points to identify the clustering structure. Proceedings of ACM SIGMOD 1999. Campello, R. J. G. B., Moulavi, D. and Sander, J. (2013). Density-based clustering based on hierarchical density estimates. PAKDD 2013, 160–172.

Ward, J. H. (1963). Hierarchical grouping to optimize an objective function. Journal of the American Statistical Association, 58(301), 236–244. Dempster, A. P., Laird, N. M. and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. Journal of the Royal Statistical Society B, 39(1), 1–38. Schwarz, G. (1978). Estimating the dimension of a model. Annals of Statistics, 6(2), 461–464. Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, 20, 53–65. Hubert, L. and Arabie, P. (1985). Comparing partitions. Journal of Classification, 2, 193–218. Aitchison, J. (1986). The Statistical Analysis of Compositional Data. Chapman and Hall.

Reference: https://scikit-learn.org/stable/modules/clustering.html ; https://scikit-learn.org/stable/modules/mixture.html